

SOFTWARE REQUIREMENTS SPECIFICATION

Employee Management System (EMS)

Web-based Employee Management System

React + TypeScript Frontend | YARP API Gateway | ASP.NET Core (.NET 9) Microservices | SQL Server Express

Document Title	Software Requirements Specification - Employee Management System
Document Version	1.0 (Draft for mentor review)
Date	28 September 2026
Prepared By	To be confirmed (student / team name)
Mentor / Reviewer	To be confirmed
Institution / Programme	To be confirmed

Document Control

Version History

Version	Date	Author	Description
1.0	28 September 2026	To be confirmed	First consolidated SRS. Combines the project requirements brief with the existing SRS v2.0 (PDF), the High Level Design diagram and the User Flow diagram.

Source Documents

Ref	Source	How it is used in this SRS
S1	Project requirements brief (technology stack, modules, roles, ports, databases, registration rule)	Primary authority. Where other sources conflict, S1 prevails.
S2	Existing SRS v2.0 (SRS.pdf)	Source of endpoint list, entity fields, business-rule wording, permissions matrix and NFR target values. Items that conflict with S1 are recorded in Appendix B.
S3	High Level Design diagram (High_Level_Design_for_EMS.png)	Source of layered architecture. Components labelled "Proposed / To be confirmed" (service discovery, Apache Kafka, containers, CI/CD) are not treated as requirements.
S4	User Flow diagram (USer_Flow_diagram.png)	Source of role-based dashboard and module flows. Flows outside S1 (forgot password, notifications, Manager dashboard) are recorded as Future Enhancements.
S5	ER diagram (ER_diagram.png)	The supplied image contains no diagram content (blank frame). The ER diagram in this SRS was regenerated from the entities in S2 (Section 11).

Requirement Status Terminology

This document does not claim that any feature is implemented merely because it is listed. The following terms are used consistently:

Term	Meaning
Existing	Present in the project artifacts (service list, ports, database names, endpoint list in S2). Whether the code fully realises it is verified during testing and is marked "To be confirmed" where relevant.
Required	The system must provide this functionality. It is a requirement, not a statement that it is already built.
Future Enhancement	Desirable but outside the current scope. Not to be treated as a requirement for acceptance.
To be confirmed	The detail is unknown or conflicting in the available material and must be decided or verified by the project team.

Table of Contents

Document Control	2
Version History	2
Source Documents	2
Requirement Status Terminology	2
Table of Contents	3
1. Introduction	6
1.1 Purpose	6
1.2 Scope	6
1.3 Intended Audience	6
1.4 Definitions	6
1.5 Acronyms and Abbreviations	6
2. Project Overview	7
2.1 Project Description	7
2.2 Problem Statement	7
2.3 Objectives	7
2.4 Proposed Solution	7
3. Scope of the System	7
3.1 In-Scope Functionality	8
3.2 Out-of-Scope Functionality	8
4. User Roles and Responsibilities	8
4.1 Permissions Summary	9
5. System Features	9
6. Functional Requirements	11
6.1 Authentication (AuthService)	11
6.2 Employee Management (EmployeeService)	14
6.3 Department Management (DepartmentService)	16
6.4 Salary Management (SalaryService)	18
6.5 Leave Management (LeaveService)	21
6.6 Attendance Management (AttendanceService)	23
6.7 Support Management (SupportService)	25
6.8 API Gateway	27
7. Non-Functional Requirements	28
7.1 Performance	28
7.2 Security	28
7.3 Reliability	28
7.4 Availability	28
7.5 Maintainability	29
7.6 Scalability	29
7.7 Usability	29

8. System Architecture	29
8.1 Overview	29
8.2 Components	29
8.3 Service Ports and Gateway Routing	30
8.4 System Architecture Diagram	30
9. Technology Stack	32
10. Database Requirements	33
10.1 Database Architecture	33
10.2 Reasons for Separate Databases	33
10.3 Consequences and Requirements	33
11. Database Design	33
11.1 Entities	34
11.2 Relationships	37
11.3 ER Diagram	38
11.4 Primary Keys, Foreign Keys and Constraints	38
11.5 Normalization	38
12. API Requirements	40
12.1 General	40
12.2 Authentication APIs (AuthService)	40
12.3 Employee APIs (EmployeeService)	40
12.4 Department APIs (DepartmentService)	40
12.5 Leave APIs (LeaveService)	41
12.6 Attendance APIs (AttendanceService)	41
12.7 Salary APIs (SalaryService)	41
12.8 Support APIs (SupportService)	42
12.9 Request and Response Content	42
13. Security Requirements	42
14. Business Rules	43
15. UI Requirements	43
16. Validation Requirements	44
17. Error Handling Requirements	44
18. Logging Requirements	45
19. Assumptions and Dependencies	45
19.1 Assumptions	45
19.2 Dependencies	45
20. System Constraints	46
21. Future Enhancements	46
22. Acceptance Criteria	46
Appendix A. Diagram Specifications	48
A.1 Placeholders	48
A.2 User Flow Diagram (supplied)	49

Appendix B. Reconciliation with Source Material49

Note: in Word, right-click the table above and choose "Update Field" (or press F9) to refresh page numbers.

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements, data model, API surface, security requirements and business rules of the Employee Management System (EMS). It is intended as the reference for design, development, testing and review of the project.

1.2 Scope

EMS is a web-based application for managing employee records, departments, salaries and payroll information, leave, attendance and internal support tickets. It consists of a React single-page frontend, a YARP API Gateway and seven ASP.NET Core Web API microservices, each with its own SQL Server database. Access is controlled by JWT authentication and role-based authorization. Detailed in-scope and out-of-scope items are given in Section 3.

1.3 Intended Audience

Audience	Use of this document
Mentor / reviewer	Assess completeness and consistency of the project specification.
Developers	Reference for service responsibilities, endpoints, entities and business rules.
Testers	Basis for test cases; requirement IDs and acceptance criteria (Section 22) support traceability.
Project team	Track which items are required, existing, future or still to be confirmed.

1.4 Definitions

Term	Definition
Employee Management System (EMS)	The system specified in this document.
API Gateway	A single entry point (YARP, port 5000) that receives frontend requests and forwards them to the appropriate backend service.
Microservice	An independently deployable backend service owning one business domain and its database.
JWT	JSON Web Token used to carry the identity and role of an authenticated user.
Role	A classification of a user (Admin, HR, Employee, Support) that determines permitted operations.
Protected endpoint	An API endpoint that requires a valid JWT and, where specified, a permitted role.
Payroll / Payslip	A monthly salary calculation for employees / an itemised salary statement for one employee in one payroll.
Support ticket	A request raised by an employee for workplace or technical assistance.
Repository pattern	A data-access layer that isolates DbContext operations from business logic.

1.5 Acronyms and Abbreviations

Acronym	Expansion	Acronym	Expansion
API	Application Programming Interface	JWT	JSON Web Token
BCrypt	Adaptive password hashing algorithm	NFR	Non-Functional Requirement
CRUD	Create, Read, Update, Delete	ORM	Object-Relational Mapper
DTO	Data Transfer Object	RBAC	Role-Based Access Control

Acronym	Expansion	Acronym	Expansion
EF Core	Entity Framework Core	REST	Representational State Transfer
EMS	Employee Management System	SPA	Single-Page Application
ER	Entity Relationship	SRS	Software Requirements Specification
FR	Functional Requirement	UI	User Interface
HR	Human Resources	YARP	Yet Another Reverse Proxy
HTTPS	HTTP over TLS	BR	Business Rule

2. Project Overview

2.1 Project Description

EMS is a web-based system that lets an organisation maintain employee information and run day-to-day HR operations from one application. Four user roles (Admin, HR, Employee, Support) use role-specific functionality. The backend is divided into domain microservices (Authentication, Employee, Department, Salary, Leave, Attendance, Support) reached only through an API Gateway.

2.2 Problem Statement

Employee records, leave requests, attendance, salary details and support requests are often handled in separate spreadsheets, e-mails or paper forms. This causes inconsistent data, no reliable history of approvals or salary changes, difficulty enforcing who may see or change sensitive data, and manual effort in calculating working hours and payroll figures.

2.3 Objectives

- Provide one web application for employee, department, salary, leave, attendance and support management.
- Enforce role-based access so that each user can perform only the operations permitted for their role.
- Protect all business APIs with JWT authentication and prevent public users from obtaining privileged roles.
- Keep an auditable history of salary revisions, leave decisions and support ticket status.
- Separate business domains into independent services with separate databases.
- Expose backend functionality to the frontend only through a single API Gateway.
- Document all APIs with Swagger / OpenAPI.

2.4 Proposed Solution

The proposed solution is a React + TypeScript frontend that calls a YARP API Gateway. The gateway routes requests to ASP.NET Core Web API services (.NET 9). Each service follows the layered structure Controller, Service, Repository, DbContext, SQL Server, and persists data with Entity Framework Core to its own SQL Server Express database. AuthService issues JWTs; other services and the gateway validate them and apply role-based authorization.

3. Scope of the System

3.1 In-Scope Functionality

Module	In-scope functionality
Authentication	Registration (Employee role by default), login, logout, JWT generation and validation, BCrypt hashing, role-based authorization, protected APIs, 401/403 handling.
Employee Management	Add, view, view profile, update, delete, search, employee details management, role-based access.
Department Management	Create, view, update, delete departments; view employees of a department; validation; deletion restrictions.
Salary Management	Salary creation, viewing, update, revision and revision history; salary components; bonuses; overtime; payroll and payslip information; role-based authorization.
Leave Management	Apply for leave, leave history, approve/reject by HR/Admin, status management, validation, prevention of self-approval.
Attendance Management	Check-in, check-out, attendance history, working-hours calculation, daily and weekly/monthly attendance information, role-based access.
Support Management	Create ticket, view tickets, assignment, status updates (In Progress, Resolved, Closed), ticket history.
API Gateway	Single entry point, service routing, centralised API access, authentication/authorization integration, hiding backend service endpoints.

3.2 Out-of-Scope Functionality

The following appear in supporting artifacts (S2-S4) but are not part of the required modules in S1. They are not acceptance criteria for this release.

- Manager role and Manager dashboard (S2, S4). Approval duties are assigned to HR/Admin.
- Project / workload allocation management (S2 FR-PRJ-001/002, BR-PRJ-001/002).
- Forgot / reset password flow and notification module including e-mail notifications (S4).
- Asynchronous messaging with Apache Kafka and service discovery (S3, labelled "Proposed / To be confirmed").
- Container orchestration, cloud deployment and CI/CD pipeline (S3); deployment approach is To be confirmed.
- Mobile or third-party clients (S3 optional client layer).
- Integration with external banking, tax or biometric systems.
- Leave balance tracking (referenced in S2/S3 but not listed in S1): To be confirmed.

4. User Roles and Responsibilities

EMS has four roles. A public visitor (unauthenticated) may only register and log in.

Role	Responsibilities	Key permissions
Admin	System owner. Manages employees, departments, salary configuration, payroll, leave decisions, ticket assignment and privileged accounts.	Full access to all modules. Creates and manages Admin/HR accounts through authorized administrative functionality.
HR	Manages workforce data and compensation; reviews leave and attendance.	Employees and departments (manage); salary, bonus, overtime, payroll (manage); approve/reject leave; view attendance and tickets.
Employee	Self-service user. Default role for public registration.	View own profile, salary, payslips and leave history; apply for leave; check-in/out; submit overtime; create and view own tickets. Cannot approve leave.

Role	Responsibilities	Key permissions
Support	Handles support tickets.	View tickets; assign tickets; update status (In Progress, Resolved, Closed); read-only employee directory access (per S2, To be confirmed).

4.1 Permissions Summary

Legend: Y = allowed, N = not allowed, Own = own records only, R = read only, TBC = To be confirmed. Consolidated from S1 and S2.

Module / Action	Admin	HR	Employee	Support
Register (public) / Login	Y	Y	Y	Y
Create Admin/HR accounts	Y	N	N	N
Employees: list / view any	Y	Y	Own	R (TBC)
Employees: add / update	Y	Y	N	N
Employees: delete	Y	TBC	N	N
Departments: view	Y	Y	Y	Y
Departments: create / update / delete	Y	Y	N	N
Salary: view own salary / payslip	Y	Y	Own	Own
Salary: create / revise / components / bonus	Y	Y	N	N
Overtime: submit	Y	Y	Y	Y
Overtime: approve / reject	Y	Y	N	N
Payroll: generate / payslip generate	Y	Y	N	N
Leave: apply / own history	Y	Y	Y	Y
Leave: approve / reject (not own)	Y	Y	N	N
Attendance: check-in / check-out / own history	Y	Y	Y	Y
Attendance: view all employees	Y	Y	N	N
Support: create ticket / view own	Y	Y	Y	Y
Support: view all tickets	Y	Y	N	Y
Support: assign / update status	Y	TBC	N	Y

5. System Features

ID	Feature	Description	Related FRs	Status
SF-01	Authentication and Authorization	Registration, login, logout, JWT, RBAC, protected APIs.	FR-AUTH-001 to 010	Required
SF-02	Employee Management	Maintain employee records, profile view, search.	FR-EMP-001 to 009	Required
SF-03	Department Management	Maintain departments and their employees.	FR-DEP-001 to 007	Required
SF-04	Salary Management	Salary, components, revisions, bonuses, overtime, payroll, payslips.	FR-SAL-001 to 011	Required
SF-05	Leave Management	Apply, review, approve/reject leave; history.	FR-LEAVE-001 to 007	Required
SF-06	Attendance Management	Check-in/out, working hours, daily/weekly/monthly information.	FR-ATT-001 to 006	Required
SF-07	Support Management	Tickets, assignment, status workflow, history.	FR-SUP-001 to 008	Required
SF-08	API Gateway	Central routing and access point for the frontend.	FR-GW-001 to 003	Required

ID	Feature	Description	Related FRs	Status
SF-09	API Documentation	Swagger / OpenAPI for services.	NFR-MAIN-003	Required
SF-10	Forgot password, notifications, Manager role, project allocation	Shown in S2/S4 only.	None	Future Enhancement

6. Functional Requirements

Each requirement uses the format: Description, Actor, Preconditions, Main Flow, Postconditions, Validation, Authorization, Expected Response and Status. All business endpoints are called through the API Gateway. Endpoint paths follow the route list in S2 (/api/v1/...); the exact gateway route prefix is To be confirmed against the implementation. Standard error responses are described in Section 17.

Common preconditions: the user is authenticated with a valid, unexpired JWT unless the requirement states the actor is a public user.

6.1 Authentication (AuthService)

FR-AUTH-001: User Registration	
Description	The system must allow a public (unauthenticated) user to create an account. The account must always be created with the Employee role.
Actor	Public user
Preconditions	The username and email are not already registered.
Main Flow	1. User submits the registration form (username, email, password; exact request fields To be confirmed). 2. AuthService validates the input. 3. AuthService assigns the Employee role, ignoring any role value in the request. 4. Password is hashed with BCrypt. 5. The user record is stored in AuthDb.
Postconditions	A Users record with role Employee exists. Whether an Employee record is created automatically in EmployeeDb: To be confirmed.
Validation	Required fields present; email format valid; username and email unique; password meets complexity rules (Section 16).
Authorization	Anonymous (no token required).
Expected Response	201 Created on success; 400 Bad Request for invalid input; 409 Conflict for duplicate username/email.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-AUTH-002: Privileged Role Protection	
Description	The system must prevent public users from registering as Admin, HR, Support or any other privileged role.
Actor	Public user, system
Preconditions	A registration request is received without an authenticated Admin context.
Main Flow	1. AuthService receives the registration request. 2. Any role supplied by the caller is discarded or rejected. 3. The account is created as Employee only.
Postconditions	No privileged account can be created through the public registration function.
Validation	Role is never taken from public input.
Authorization	Anonymous.
Expected Response	201 Created with Employee role, or 400/403 if a privileged role is explicitly requested (exact behaviour To be confirmed).
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-AUTH-003: Admin/HR Account Management	
Description	Admin and HR accounts (and other privileged roles) must be created and managed only through authorized administrative functionality.
Actor	Admin
Preconditions	The caller is authenticated as Admin.
Main Flow	1. Admin submits a request to create or change a user role. 2. The system verifies the Admin role. 3. The account or role is created/updated.

Postconditions	A privileged account exists or a role is changed by an authorized user.
Validation	Same field and password rules as registration; role must be one of Admin, HR, Employee, Support.
Authorization	Admin only.
Expected Response	201 Created or 200 OK; 401; 403 for non-Admin callers.
Status	Required. The endpoint for this function is not documented (S2 lists only POST /auth/register, described there as admin-only); endpoint and design: To be confirmed.

FR-AUTH-004: User Login

Description	The system must authenticate users using credentials and return a JWT.
Actor	Public user (any role)
Preconditions	The user has a registered account.
Main Flow	1. User submits username/email and password (login identifier To be confirmed). 2. AuthService retrieves the user and verifies the password against the BCrypt hash. 3. On success, a JWT is generated and returned. 4. The frontend stores the token and redirects to the role-based dashboard.
Postconditions	The user holds a valid token for subsequent API calls.
Validation	Credentials required; no detail revealing which field was wrong.
Authorization	Anonymous.
Expected Response	200 OK with token; 400 for missing input; 401 Unauthorized for invalid credentials.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-AUTH-005: User Logout

Description	The system must allow a logged-in user to end the session.
Actor	Admin, HR, Employee, Support
Preconditions	The user is logged in.
Main Flow	1. User selects Logout. 2. The frontend clears the stored token and session state. 3. The user is redirected to the login page.
Postconditions	The frontend no longer sends the token. Server-side token revocation is not defined.
Validation	Not applicable.
Authorization	Authenticated user.
Expected Response	No API call is required for client-side logout. A logout endpoint and token invalidation: To be confirmed.
Status	Required. Client-side behaviour per S4; server-side logout: To be confirmed.

FR-AUTH-006: JWT Token Generation

Description	The system must issue a signed JWT after successful login containing identity and role claims.
Actor	System (AuthService)
Preconditions	Credentials have been verified.
Main Flow	1. AuthService builds claims (user id, role, email, employeeId per S2). 2. Token is signed using the configured secret/key and given an expiry time. 3. Token is returned in the login response.
Postconditions	A signed token with an expiry exists.
Validation	Signing key must be present in secure configuration.
Authorization	Internal to login.
Expected Response	Token in 200 OK login response. Signing algorithm (HMAC-SHA256 or RSA) and token lifetime: To be confirmed.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-AUTH-007: JWT Token Validation

Description	The gateway and each protected service must validate the JWT on every protected request.
Actor	System (Gateway, services)
Preconditions	A request carries Authorization: Bearer <token>.
Main Flow	1. Request reaches the gateway/service. 2. Signature, issuer/audience (if configured) and expiry are validated. 3. Claims are made available to authorization checks. 4. Invalid or missing tokens are rejected.
Postconditions	Only requests with valid tokens reach protected controllers.
Validation	Token must be well-formed, correctly signed and unexpired.
Authorization	All protected endpoints.
Expected Response	401 Unauthorized when missing, malformed or expired.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-AUTH-008: Role-Based Authorization

Description	The system must restrict each endpoint to the roles permitted for it.
Actor	System
Preconditions	A valid token with a role claim is presented.
Main Flow	1. The role claim is read from the token. 2. It is compared with the roles allowed for the endpoint. 3. The request proceeds or is denied.
Postconditions	Users can access only functionality permitted for their role.
Validation	Role must be one of Admin, HR, Employee, Support.
Authorization	As per the permission summary in Section 4.1.
Expected Response	403 Forbidden when the role is not permitted.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-AUTH-009: Unauthorized and Forbidden Access Handling

Description	The system must return consistent responses for unauthenticated (401) and unauthorized (403) requests and the frontend must handle them.
Actor	System, frontend
Preconditions	A protected endpoint is called without a valid token or without sufficient role.
Main Flow	1. The API returns 401 or 403. 2. On 401 the frontend clears the session and redirects to login. 3. On 403 the frontend shows an access-denied message and keeps the session.
Postconditions	No protected data is returned; the user receives a clear outcome.
Validation	Not applicable.
Authorization	All protected endpoints.
Expected Response	401 Unauthorized; 403 Forbidden (error body format in Section 17).
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-AUTH-010: Password Hashing

Description	The system must store passwords only as BCrypt hashes.
Actor	System (AuthService)
Preconditions	A password is supplied during registration or account creation.
Main Flow	1. Password is hashed with BCrypt. 2. Only the hash is stored in Users.PasswordHash. 3. At login the supplied password is verified against the hash.
Postconditions	No plaintext password is stored or logged.
Validation	Password complexity rules (Section 16).

Authorization	Internal.
Expected Response	Not directly visible; failures produce 400 (validation) or 401 (login).
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

6.2 Employee Management (EmployeeService)

FR-EMP-001: Add Employee	
Description	The system must allow authorized users to create an employee record.
Actor	Admin, HR
Preconditions	Caller authenticated as Admin or HR; the linked user identity exists (UserId).
Main Flow	1. Actor opens the add-employee form and enters details. 2. Frontend calls POST /api/v1/employees. 3. EmployeeService validates and saves the Employee record.
Postconditions	A new Employee record exists in EmployeeDb.
Validation	Required fields (first name, last name, email, department, date of joining); unique email; joining date not in the future; department must exist.
Authorization	Admin, HR.
Expected Response	201 Created; 400; 401; 403; 409 for duplicate email.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-EMP-002: View Employees	
Description	The system must list employee records for authorized roles.
Actor	Admin, HR (Support read-only: To be confirmed)
Preconditions	Caller authenticated with a permitted role.
Main Flow	1. Actor opens the employee directory. 2. Frontend calls GET /api/v1/employees. 3. The list is returned and displayed.
Postconditions	Employee list displayed.
Validation	Not applicable.
Authorization	Admin, HR; Support read: To be confirmed.
Expected Response	200 OK with list; 401; 403.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-EMP-003: View Employee by ID	
Description	The system must return a single employee record by identifier.
Actor	Admin, HR
Preconditions	Employee identifier is known.
Main Flow	1. Actor selects an employee. 2. Frontend calls GET /api/v1/employees/{id}. 3. Details are shown.
Postconditions	Details displayed.
Validation	Identifier must be a valid UUID.
Authorization	Admin, HR.
Expected Response	200 OK; 401; 403; 404 Not Found.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-EMP-004: View Employee Profile	
Description	An authenticated user must be able to view their own employee profile.

Actor	Admin, HR, Employee, Support
Preconditions	The user has a linked employee record.
Main Flow	1. User opens the profile page. 2. Frontend calls GET /api/v1/employees/me. 3. The profile is returned based on the token identity.
Postconditions	Own profile displayed; no other employee data exposed.
Validation	Identity is taken from the token, not from user input.
Authorization	Any authenticated user.
Expected Response	200 OK; 401; 404 if no employee record is linked.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-EMP-005: Update Employee

Description	The system must allow authorized users to change employee details.
Actor	Admin, HR
Preconditions	The employee exists.
Main Flow	1. Actor edits fields in the form. 2. Frontend calls PUT /api/v1/employees/{id}. 3. Service validates and saves changes.
Postconditions	Record updated; UpdatedAt changed.
Validation	Same rules as FR-EMP-001; email uniqueness excluding the current record.
Authorization	Admin, HR.
Expected Response	200 OK (or 204 No Content); 400; 401; 403; 404; 409.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-EMP-006: Delete Employee

Description	The system must allow authorized users to delete an employee record.
Actor	Admin (HR: To be confirmed)
Preconditions	The employee exists.
Main Flow	1. Actor selects Delete and confirms in a dialog. 2. Frontend calls DELETE /api/v1/employees/{id}. 3. Service removes (or marks) the record.
Postconditions	Employee no longer appears in listings. Whether deletion is soft or hard, and handling of related history in other services, is To be confirmed (S2 BR-EMP-003 states related history is archived).
Validation	Identifier must exist.
Authorization	Admin; HR To be confirmed.
Expected Response	204 No Content (or 200); 401; 403; 404.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-EMP-007: Search Employee

Description	The system must allow searching employees by a text term over name, role and email.
Actor	Admin, HR
Preconditions	Caller authenticated with a permitted role.
Main Flow	1. Actor enters a search term. 2. Frontend calls GET /api/v1/employees/search with the term (parameter name To be confirmed). 3. Matching employees are returned.
Postconditions	Filtered list shown.
Validation	Term is length-limited and treated as data (EF Core parameterised queries).
Authorization	Admin, HR.
Expected Response	200 OK (empty list if no match); 401; 403.

Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.
--------	--

FR-EMP-008: Employee Details Management	
Description	The system must maintain employee details: name, email, phone, date of joining, department, role and employment status. Employees may edit only the restricted personal information allowed for self-service.
Actor	Admin, HR; Employee (limited)
Preconditions	Employee record exists.
Main Flow	1. Authorized actor edits details through FR-EMP-005. 2. Employee self-service editing is limited to permitted fields.
Postconditions	Details are consistent with validation rules.
Validation	As FR-EMP-001. Which fields an Employee may self-edit: To be confirmed.
Authorization	Admin, HR full; Employee limited.
Expected Response	200 OK; 400; 403.
Status	Required. Not documented in existing project artifacts; design and implementation: To be confirmed.

FR-EMP-009: Employee Module Access Control	
Description	The system must restrict employee-management operations to authorized roles.
Actor	System
Preconditions	Any employee endpoint is called.
Main Flow	1. Token and role are validated. 2. Admin/HR may manage; Employee may access only /me; others denied.
Postconditions	Unauthorized operations are blocked.
Validation	Not applicable.
Authorization	See Section 4.1.
Expected Response	401 or 403.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

6.3 Department Management (DepartmentService)

FR-DEP-001: Create Department	
Description	The system must allow authorized users to create a department.
Actor	Admin, HR
Preconditions	Caller authenticated as Admin or HR.
Main Flow	1. Actor enters department name and description. 2. Frontend calls POST /api/v1/departments. 3. Service validates and saves.
Postconditions	Department stored in DepartmentDb.
Validation	Name required and unique (BR-DEP-001).
Authorization	Admin, HR.
Expected Response	201 Created; 400; 401; 403; 409 for duplicate name.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-DEP-002: View Departments	
Description	The system must list departments to any authenticated user.
Actor	Admin, HR, Employee, Support
Preconditions	Authenticated.
Main Flow	1. User opens the department list. 2. Frontend calls GET /api/v1/departments.

Postconditions	List displayed.
Validation	Not applicable.
Authorization	Any authenticated user.
Expected Response	200 OK; 401.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-DEP-003: View Department by ID

Description	The system must return one department by identifier.
Actor	Any authenticated user
Preconditions	Department id known.
Main Flow	1. Frontend calls GET /api/v1/departments/{id}.
Postconditions	Details displayed.
Validation	Valid UUID.
Authorization	Any authenticated user.
Expected Response	200 OK; 401; 404.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-DEP-004: Update Department

Description	The system must allow authorized users to change department name and description.
Actor	Admin, HR
Preconditions	Department exists.
Main Flow	1. Actor edits fields. 2. Frontend calls PUT /api/v1/departments/{id}.
Postconditions	Department updated.
Validation	Name required and unique excluding current record.
Authorization	Admin, HR.
Expected Response	200 OK (or 204); 400; 401; 403; 404; 409.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-DEP-005: Delete Department

Description	The system must allow deletion of a department only when business rules permit.
Actor	Admin, HR
Preconditions	Department exists.
Main Flow	1. Actor selects Delete and confirms. 2. Frontend calls DELETE /api/v1/departments/{id}. 3. The system checks whether employees are assigned (BR-DEP-002). 4. If none, the department is deleted; otherwise the request is refused.
Postconditions	Department removed, or unchanged with an explanatory error.
Validation	No active employees assigned. Method of checking employees across services (service call or other): To be confirmed.
Authorization	Admin, HR.
Expected Response	204 No Content (or 200); 401; 403; 404; refusal status 409 Conflict or 400 (To be confirmed).
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-DEP-006: View Employees of a Department

Description	The system must show the employees associated with a department.
Actor	Admin, HR
Preconditions	Department exists.

Main Flow	1. Actor opens a department. 2. The employee list for that department is retrieved (e.g. from EmployeeService using DepartmentId). 3. Employees and count are displayed.
Postconditions	Associated employees displayed.
Validation	Valid department id.
Authorization	Admin, HR.
Expected Response	200 OK; 401; 403; 404.
Status	Required. Not documented in existing project artifacts; design and implementation: To be confirmed. No dedicated endpoint is listed in S2.

FR-DEP-007: Department Validation	
Description	The system must validate department input and references (unique name, required fields; employees reference existing departments).
Actor	System
Preconditions	Create/update department or employee request.
Main Flow	1. Input is validated by the service. 2. Violations return a descriptive error.
Postconditions	Only valid department data is stored.
Validation	Name required, unique; length limits To be confirmed.
Authorization	Same as parent operation.
Expected Response	400 or 409.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

6.4 Salary Management (SalaryService)

FR-SAL-001: Create Employee Salary	
Description	The system must allow authorized users to create the base salary record for an employee.
Actor	Admin, HR
Preconditions	Employee exists; no salary record exists yet (EmployeeId is unique in EmployeeSalaries).
Main Flow	1. Actor enters employee, basic salary and effective date. 2. Frontend calls POST /api/v1/salaries. 3. Service validates and saves.
Postconditions	EmployeeSalaries record created.
Validation	BasicSalary must not be negative and must be a positive non-zero value per S2 BR-SAL-001 (zero handling To be confirmed); effective date required.
Authorization	Admin, HR.
Expected Response	201 Created; 400; 401; 403; 409 if a salary already exists.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-SAL-002: View Salary	
Description	The system must allow users to view salary information: Admin/HR for any employee, other users for their own only.
Actor	Admin, HR, Employee, Support
Preconditions	Salary exists.
Main Flow	1. User opens the salary page. 2. Frontend calls GET /api/v1/salaries/{empId}. 3. Service checks that the caller is Admin/HR or the owner.
Postconditions	Salary shown.
Validation	Valid employee id; ownership check.
Authorization	Admin, HR any; others own only.

Expected Response	200 OK; 401; 403; 404.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-SAL-003: Update Salary

Description	The system must allow authorized users to update the base salary record.
Actor	Admin, HR
Preconditions	Salary record exists.
Main Flow	1. Actor changes salary values. 2. Frontend calls PUT /api/v1/salaries/{empld}.
Postconditions	Record updated. Changes to the salary amount should be captured as a revision (FR-SAL-004).
Validation	Salary not negative; required fields.
Authorization	Admin, HR.
Expected Response	200 OK (or 204); 400; 401; 403; 404.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-SAL-004: Salary Revision

Description	The system must record a salary revision with previous salary, revised salary, date and reason.
Actor	Admin, HR
Preconditions	Salary record exists.
Main Flow	1. Actor enters revised salary and reason. 2. Frontend calls POST /api/v1/salaries/{empld}/revisions. 3. Service stores the revision (append-only).
Postconditions	SalaryRevisions record added; existing revisions unchanged.
Validation	Revised salary not negative; reason required (To be confirmed).
Authorization	Admin, HR.
Expected Response	201 Created; 400; 401; 403; 404.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-SAL-005: Salary Revision History

Description	The system must list all revisions for an employee.
Actor	Admin, HR; Employee (own, To be confirmed)
Preconditions	Employee exists.
Main Flow	1. Actor opens revision history. 2. Frontend calls GET /api/v1/salaries/{empld}/revisions.
Postconditions	Revisions displayed.
Validation	Valid employee id.
Authorization	Admin, HR; own records for Employee (To be confirmed).
Expected Response	200 OK; 401; 403; 404.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-SAL-006: Salary Components

Description	The system must maintain salary components (name, amount, type Earning or Deduction) per employee.
Actor	Admin, HR (view: Employee own)
Preconditions	Salary record exists.
Main Flow	1. Actor adds a component via POST /api/v1/salaries/{empld}/components. 2. Components listed via GET /api/v1/salaries/{empld}/components. 3. A component is changed via PUT /api/v1/salary-components/{id}.
Postconditions	Components stored and used in payroll totals.

Validation	Name required; amount not negative; type must be Earning or Deduction.
Authorization	Admin, HR modify; Employee view own.
Expected Response	201 Created / 200 OK; 400; 401; 403; 404.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-SAL-007: Bonus Management

Description	The system must record and list bonuses (type, amount, date, reason) for an employee.
Actor	Admin, HR
Preconditions	Employee salary exists.
Main Flow	1. Actor grants a bonus via POST /api/v1/salaries/{empld}/bonuses. 2. Bonuses listed via GET /api/v1/salaries/{empld}/bonuses. 3. A bonus is edited via PUT /api/v1/bonuses/{id}.
Postconditions	Bonus stored; included in payroll totals.
Validation	Amount not negative; bonus date required.
Authorization	Admin, HR.
Expected Response	201 Created / 200 OK; 400; 401; 403; 404.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-SAL-008: Overtime Management

Description	Employees must be able to submit overtime; authorized users approve or reject it. Overtime amount = Hours x Rate.
Actor	Employee (submit), Admin/HR (approve/reject)
Preconditions	Employee exists.
Main Flow	1. Employee submits overtime via POST /api/v1/overtime (status Pending). 2. Admin/HR reviews the record (GET /api/v1/overtime/{empld}). 3. Admin/HR calls PUT /api/v1/overtime/{id}/approve or /reject. 4. Only Approved overtime is included in payroll.
Postconditions	Overtime status is Approved or Rejected with ApprovedBy and ApprovedAt.
Validation	Hours greater than zero; rate not negative; status must be Pending to be decided.
Authorization	Submit: authenticated; approve/reject: Admin, HR.
Expected Response	201 Created / 200 OK; 400; 401; 403; 404; 409 if already decided.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-SAL-009: Payroll Generation

Description	The system must generate payroll for a given month and year, producing a payroll item per employee (basic salary, components, bonus, overtime, deductions, net salary).
Actor	Admin, HR
Preconditions	Employee salary data exists.
Main Flow	1. Actor selects month and year. 2. Frontend calls POST /api/v1/payroll. 3. Service calculates items using: Net = Basic + Earnings + Bonuses + Approved Overtime - Deductions. 4. Payroll and PayrollItems are stored; results viewable via GET /api/v1/payroll, /{id}, /{id}/items.
Postconditions	Payroll and items exist and are not recalculated once finalised.
Validation	Valid month (1-12) and year; no duplicate payroll for the same period (To be confirmed).
Authorization	Admin, HR.
Expected Response	201 Created; 400; 401; 403; 409.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-SAL-010: Payslip Information

Description	The system must generate itemised payslips from payroll and allow authorized viewing.
--------------------	---

Actor	Admin, HR (generate, view all); Employee (view own)
Preconditions	Payroll and payroll item exist.
Main Flow	- Admin/HR calls POST /api/v1/payslips/{payId}/{empld}. - A payslip with a unique PayslipNumber is stored. - Users view via GET /api/v1/payslips and GET /api/v1/payslips/{id}.
Postconditions	Payslip available to the owning employee.
Validation	Payroll item must exist; one payslip per employee per payroll (To be confirmed).
Authorization	Generate: Admin, HR; view: Admin, HR, and owner.
Expected Response	201 Created / 200 OK; 401; 403; 404; 409.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-SAL-011: Salary Authorization

Description	Only authorized roles may create or modify salary, components, revisions, bonuses, overtime decisions, payroll and payslips; other users may view only their own salary data.
Actor	System
Preconditions	Any salary endpoint is called.
Main Flow	- JWT and role are validated. - Ownership is checked for read endpoints of non-Admin/HR users.
Postconditions	Unauthorized changes and cross-employee reads are prevented.
Validation	Not applicable.
Authorization	See Section 4.1.
Expected Response	401 or 403.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

6.5 Leave Management (LeaveService)

FR-LEAVE-001: Apply for Leave

Description	An employee must be able to submit a leave request.
Actor	Employee (all authenticated roles)
Preconditions	Employee record exists.
Main Flow	- Employee enters leave type, start date, end date and reason. - Frontend calls POST /api/v1/leaves. - Service validates and stores the request with status Pending.
Postconditions	Leave stored with Pending status and AppliedAt.
Validation	Required fields; StartDate on or before EndDate (BR-LEAVE-001); no overlap with existing leave (BR-LEAVE-002).
Authorization	Any authenticated user for own leave.
Expected Response	201 Created; 400; 401; 409 for overlap.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-LEAVE-002: View Leave History

Description	An employee must be able to view their own leave history; HR/Admin may view any employee.
Actor	Employee, HR, Admin
Preconditions	Authenticated.
Main Flow	- User opens leave history. - Frontend calls GET /api/v1/leaves/employee/{empld} or GET /api/v1/leaves/{empld}/history. - Single record via GET /api/v1/leaves/{id}.
Postconditions	History displayed.
Validation	Ownership check for Employee role.
Authorization	Owner, HR, Admin.

Expected Response	200 OK; 401; 403; 404.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-LEAVE-003: Approve Leave

Description	HR or Admin must be able to approve a pending leave request.
Actor	HR, Admin
Preconditions	Leave exists with status Pending.
Main Flow	1. Reviewer opens the pending request. 2. Frontend calls PUT /api/v1/leaves/{id}/approve. 3. Service checks the reviewer is not the leave owner. 4. Status becomes Approved; ApprovedBy and ApprovedAt are recorded.
Postconditions	Leave status Approved.
Validation	Status must be Pending; reviewer not the applicant (BR-LEAVE-003).
Authorization	HR, Admin.
Expected Response	200 OK; 400; 401; 403; 404; 409 if already decided.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-LEAVE-004: Reject Leave

Description	HR or Admin must be able to reject a pending leave request.
Actor	HR, Admin
Preconditions	Leave exists with status Pending.
Main Flow	1. Reviewer calls PUT /api/v1/leaves/{id}/reject. 2. Service applies the same checks as approval. 3. Status becomes Rejected with reviewer and time recorded.
Postconditions	Leave status Rejected.
Validation	As FR-LEAVE-003. Rejection reason field: To be confirmed.
Authorization	HR, Admin.
Expected Response	200 OK; 400; 401; 403; 404; 409.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-LEAVE-005: Leave Status Management

Description	The system must manage leave status with the values Pending, Approved and Rejected.
Actor	System
Preconditions	A leave request exists.
Main Flow	1. New leave starts as Pending. 2. Only Pending can change, to Approved or Rejected. 3. Decided leave cannot be changed by a further decision.
Postconditions	Status is always one of the defined values.
Validation	Allowed transitions only.
Authorization	Transitions by HR, Admin.
Expected Response	409 or 400 for invalid transitions.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-LEAVE-006: Leave Validation

Description	The system must validate leave dates and overlaps before saving.
Actor	System
Preconditions	A leave request is submitted.
Main Flow	1. Required fields checked.

	2. StartDate <= EndDate checked. 3. Overlap with the same employee's existing leave checked.
Postconditions	Only valid leave is stored.
Validation	BR-LEAVE-001, BR-LEAVE-002. Rule on past dates: To be confirmed.
Authorization	As FR-LEAVE-001.
Expected Response	400 Bad Request; 409 Conflict for overlap.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-LEAVE-007: Prevent Self-Approval

Description	The system must prevent an employee from approving or rejecting their own leave, including HR/Admin users.
Actor	System
Preconditions	A decision request is received.
Main Flow	1. The reviewer identity (from token) is compared with the leave owner. 2. If equal, the decision is refused.
Postconditions	Own leave stays Pending.
Validation	Reviewer id must differ from leave EmployeeId.
Authorization	HR, Admin (excluding own leave).
Expected Response	403 Forbidden (or 400; To be confirmed).
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

6.6 Attendance Management (AttendanceService)

FR-ATT-001: Employee Check-In

Description	An employee must be able to record a check-in for the current day.
Actor	Any authenticated user with an employee record
Preconditions	No check-in exists for the employee on this date.
Main Flow	1. Employee selects Check In. 2. Frontend calls POST /api/v1/attendance/check-in. 3. Service stores CheckInTime with status CheckedIn.
Postconditions	Attendance record created for the date.
Validation	One check-in per employee per date (BR-ATT-001).
Authorization	Any authenticated user.
Expected Response	201 Created; 401; 409 if already checked in.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-ATT-002: Employee Check-Out

Description	An employee must be able to record a check-out for an open attendance record.
Actor	Any authenticated user with an employee record
Preconditions	An open check-in exists.
Main Flow	1. Employee selects Check Out. 2. Frontend calls PUT /api/v1/attendance/{id}/check-out. 3. Service stores CheckOutTime and sets status CheckedOut.
Postconditions	Record completed.
Validation	Record must belong to the caller; CheckOutTime after CheckInTime (BR-ATT-002); not already checked out.
Authorization	Owner.
Expected Response	200 OK; 400; 401; 403; 404; 409.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-ATT-003: Working Hours Calculation	
Description	The system must calculate working duration as CheckOutTime minus CheckInTime.
Actor	System
Preconditions	A completed attendance record exists.
Main Flow	- Duration is computed on check-out or when records are read. - Duration is shown per day and in totals.
Postconditions	Working hours available for display. Whether stored or computed on read: To be confirmed (no duration field in S2 entity).
Validation	Only records with check-out are counted.
Authorization	As the record access rules.
Expected Response	Included in 200 OK responses.
Status	Required. Not documented in existing project artifacts; design and implementation: To be confirmed.

FR-ATT-004: Daily Attendance and History	
Description	Users must be able to view daily attendance and their attendance history; HR/Admin may view all employees.
Actor	All roles (own); Admin, HR (all)
Preconditions	Authenticated.
Main Flow	- User opens attendance. - Frontend calls GET /api/v1/attendance/employee/{empld} for own history, GET /api/v1/attendance for all (Admin/HR), or GET /api/v1/attendance/{id} for one record.
Postconditions	Records displayed.
Validation	Ownership check for non-Admin/HR.
Authorization	Own: any; all: Admin, HR.
Expected Response	200 OK; 401; 403; 404.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-ATT-005: Weekly and Monthly Attendance Information	
Description	The system must present weekly and monthly attendance information (days present, total hours).
Actor	All roles (own); Admin, HR (team)
Preconditions	Attendance records exist.
Main Flow	- User selects week or month. - Records for the period are retrieved and summarised.
Postconditions	Summary displayed.
Validation	Valid date range.
Authorization	As FR-ATT-004.
Expected Response	200 OK. Whether done by a dedicated endpoint or in the frontend: To be confirmed (none listed in S2).
Status	Required. Not documented in existing project artifacts; design and implementation: To be confirmed.

FR-ATT-006: Role-Based Attendance Access	
Description	The system must allow employees to access only their own attendance and only Admin/HR to access all attendance.
Actor	System
Preconditions	Attendance endpoint called.
Main Flow	- Role and identity are read from the token. - Access is allowed or denied.
Postconditions	No cross-employee attendance disclosure.
Validation	Not applicable.
Authorization	See Section 4.1.
Expected Response	401 or 403.

Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.
--------	--

6.7 Support Management (SupportService)

FR-SUP-001: Create Support Ticket	
Description	An employee must be able to create a support ticket.
Actor	Any authenticated user
Preconditions	Employee record exists.
Main Flow	1. User enters title, description and priority (Low, Medium, High). 2. Frontend calls POST /api/v1/support-tickets. 3. Service stores the ticket with status Open.
Postconditions	Ticket created with status Open.
Validation	Title and description required; priority must be a defined value.
Authorization	Any authenticated user.
Expected Response	201 Created; 400; 401.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-SUP-002: View Own Tickets	
Description	A user must be able to view their own tickets.
Actor	Any authenticated user
Preconditions	Authenticated.
Main Flow	1. Frontend calls GET /api/v1/support-tickets/employee/{id}. 2. Individual ticket via GET /api/v1/support-tickets/{id}.
Postconditions	Own tickets displayed.
Validation	Ownership check.
Authorization	Owner.
Expected Response	200 OK; 401; 403; 404.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-SUP-003: View All Tickets	
Description	HR and Support (and Admin) must be able to view all tickets.
Actor	HR, Support, Admin
Preconditions	Authenticated with permitted role.
Main Flow	1. Frontend calls GET /api/v1/support-tickets. 2. Queue is displayed with status counts.
Postconditions	All tickets displayed.
Validation	Not applicable.
Authorization	HR, Support, Admin.
Expected Response	200 OK; 401; 403.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-SUP-004: Assign Ticket	
Description	Authorized staff must be able to assign a ticket to an operator.
Actor	Support, Admin (HR: To be confirmed)
Preconditions	Ticket exists and is not Closed.
Main Flow	1. Actor selects an assignee. 2. Frontend calls PUT /api/v1/support-tickets/{id}. 3. Service stores AssignedTo and sets status Assigned.

Postconditions	Ticket assigned.
Validation	Assignee must be a Support or Admin user (BR-SUP-002).
Authorization	Support, Admin.
Expected Response	200 OK; 400; 401; 403; 404.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-SUP-005: Update Ticket Status (In Progress)

Description	Authorized staff must be able to mark a ticket In Progress.
Actor	Support, Admin
Preconditions	Ticket is Open or Assigned.
Main Flow	1. Actor sets status InProgress. 2. Frontend calls PUT /api/v1/support-tickets/{id}.
Postconditions	Status InProgress.
Validation	Transition allowed by BR-SUP-001.
Authorization	Support, Admin.
Expected Response	200 OK; 400 for invalid transition; 401; 403; 404.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-SUP-006: Resolve Ticket

Description	Authorized staff must be able to resolve a ticket.
Actor	Support, Admin
Preconditions	Ticket is InProgress.
Main Flow	1. Actor sets status Resolved. 2. Service stores ResolvedAt.
Postconditions	Status Resolved; ResolvedAt set.
Validation	Transition allowed by BR-SUP-001.
Authorization	Support, Admin.
Expected Response	200 OK; 400; 401; 403; 404.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-SUP-007: Close Ticket

Description	A resolved ticket must be closable. Who may close (staff, owner or both): To be confirmed.
Actor	Support, Admin (owner: To be confirmed)
Preconditions	Ticket is Resolved.
Main Flow	1. Actor sets status Closed via PUT /api/v1/support-tickets/{id}.
Postconditions	Status Closed; no further changes.
Validation	Transition allowed by BR-SUP-001.
Authorization	Support, Admin.
Expected Response	200 OK; 400; 401; 403; 404.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-SUP-008: Ticket History

Description	Users must be able to see a ticket's status progression over time.
Actor	Owner, HR, Support, Admin
Preconditions	Ticket exists.
Main Flow	1. User opens a ticket. 2. Current status, assignee and timestamps are displayed.

Postconditions	Ticket information displayed.
Validation	Ownership check for Employee.
Authorization	Owner; HR, Support, Admin.
Expected Response	200 OK.
Status	Required. Not documented in existing project artifacts; design and implementation: To be confirmed. S2 defines no history table; only current status, CreatedAt, UpdatedAt and ResolvedAt exist. A ticket-history entity would be a new design decision.

6.8 API Gateway

FR-GW-001: Central Entry Point and Routing	
Description	The YARP API Gateway (port 5000) must be the only endpoint used by the frontend and must route requests to the correct service by route prefix.
Actor	System
Preconditions	Gateway and target service are running.
Main Flow	1. Frontend sends a request to the gateway. 2. Gateway matches the route and forwards to the service (Section 8.3). 3. Response is returned to the frontend.
Postconditions	Frontend never calls service ports directly.
Validation	Unknown routes are not forwarded.
Authorization	Per route.
Expected Response	Service response is passed through; 404 for unmatched routes; 502/503 if a service is unavailable (To be confirmed).
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-GW-002: Authentication and Authorization Integration	
Description	The gateway must integrate with JWT authentication so that protected routes reject invalid tokens. Whether the gateway also enforces role policies per route or only services do: To be confirmed.
Actor	System
Preconditions	JWT signing configuration shared securely.
Main Flow	1. Gateway validates the token for protected routes. 2. Valid requests are forwarded with the Authorization header.
Postconditions	Unauthenticated requests do not reach protected services.
Validation	Token validity.
Authorization	Login and register routes are anonymous.
Expected Response	401 Unauthorized; 403 Forbidden.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

FR-GW-003: Backend Endpoint Hiding	
Description	Individual service URLs and ports must not be exposed to the frontend; the frontend uses only the gateway base address.
Actor	System, frontend
Preconditions	Frontend configured with the gateway address.
Main Flow	1. Frontend base URL points to the gateway. 2. Service addresses exist only in gateway configuration.
Postconditions	Backend layout hidden from clients.
Validation	Not applicable.
Authorization	Not applicable.
Expected Response	Not applicable. Network-level blocking of direct service access: To be confirmed.
Status	Required. Documented in existing SRS v2.0 / project artifacts; implementation verification: To be confirmed.

7. Non-Functional Requirements

Numeric targets are taken from the existing SRS v2.0 (S2) as design targets. They must be validated by testing; they are not claims of measured performance.

7.1 Performance

ID	Requirement	Measure / Verification
NFR-PERF-001	Normal API requests through the gateway should be answered within an acceptable time under expected load.	Target: 95% of requests under 200 ms at normal load (S2). Verified by load test; tool To be confirmed.
NFR-PERF-002	The system should support concurrent users without noticeable degradation.	Target: 500 concurrent sessions (S2). Verified by load test.
NFR-PERF-003	List and search endpoints should not load unnecessary data.	Pagination for large lists: To be confirmed.

7.2 Security

ID	Requirement	Measure / Verification
NFR-SEC-001	All protected endpoints must require a valid JWT.	Requests without token return 401 (test).
NFR-SEC-002	Endpoints must enforce role-based authorization.	Wrong-role requests return 403 (test).
NFR-SEC-003	Passwords must be stored only as BCrypt hashes.	Database inspection shows no plaintext.
NFR-SEC-004	Secrets (JWT key, connection strings) must not be hard-coded in source control.	Configuration review; secure storage mechanism To be confirmed.
NFR-SEC-005	Data access must use parameterised queries through EF Core.	Code review; SQL injection tests.
NFR-SEC-006	Deployed environments should use HTTPS.	S2 states TLS 1.3 as a target; the local development setup uses HTTP. Deployment TLS settings: To be confirmed.

7.3 Reliability

ID	Requirement	Measure / Verification
NFR-REL-001	A failure in one service must not stop unrelated services.	Stop one service in test; others continue to respond.
NFR-REL-002	Database changes that span several tables (payroll generation) must be atomic.	Transaction test for payroll generation.
NFR-REL-003	The gateway must return a clear error when a target service is unavailable.	Failure test; status code To be confirmed.

7.4 Availability

ID	Requirement	Measure / Verification
NFR-AVAIL-001	The system should be available during business hours.	Target: 99.9% (S2). For a training project this is a design target; how it is measured is To be confirmed.

7.5 Maintainability

ID	Requirement	Measure / Verification
NFR-MAIN-001	Each service must follow Controller, Service, Repository, DbContext layering where applicable.	Code review.
NFR-MAIN-002	REST conventions and naming must be consistent across services.	API review against Section 12.
NFR-MAIN-003	Each service must expose Swagger / OpenAPI documentation.	Swagger UI reachable per service.
NFR-MAIN-004	Each service must have its own EF Core migrations and database.	Migration files exist per service.

7.6 Scalability

ID	Requirement	Measure / Verification
NFR-SCAL-001	Services must be independently deployable so that a busy service can be scaled separately.	Architecture review. Actual scaling infrastructure: To be confirmed.
NFR-SCAL-002	The gateway must not hold per-user session state.	Stateless JWT design review.

7.7 Usability

ID	Requirement	Measure / Verification
NFR-USE-001	The UI must be usable on current desktop browsers and be responsive.	Manual test on Chromium, Firefox, Edge (Safari To be confirmed).
NFR-USE-002	Validation errors must be shown next to the relevant field with a clear message.	UI test.
NFR-USE-003	Users must see only navigation items permitted for their role.	UI test for each role.
NFR-USE-004	S2 targets WCAG 2.1 Level AA.	Full compliance To be confirmed; accessibility review recommended.

8. System Architecture

8.1 Overview

EMS uses a microservices architecture. The React frontend communicates only with the YARP API Gateway. The gateway routes to seven services. Each service owns its database and uses Entity Framework Core.

Request flow: Frontend > API Gateway > Backend Service > Entity Framework Core > SQL Server.

Layered structure inside each service where applicable: Controller > Service > Repository > DbContext > SQL Server.

8.2 Components

Component	Responsibility
React frontend	Single-page UI, role-based routing (React Router), calls gateway only.
API Gateway (YARP)	Central entry point, request routing, JWT/authorization integration, hides service endpoints.
AuthService	Registration, login, password hashing, JWT issuing. Database: AuthDb.
EmployeeService	Employee records, profile, search. Database: EmployeeDb.
DepartmentService	Department records. Database: DepartmentDb.
SalaryService	Salary, components, revisions, bonuses, overtime, payroll, payslips. Database: SalaryDb.
LeaveService	Leave requests and decisions. Database: LeaveDb.

Component	Responsibility
AttendanceService	Check-in/out and attendance records. Database: AttendanceDb.
SupportService	Support tickets. Database: SupportDb.

8.3 Service Ports and Gateway Routing

Component	Port	Database	Gateway route prefix (per S2, To be confirmed)
API Gateway	5000	-	/api/v1/*
AuthService	5088	AuthDb	/api/v1/auth/*
EmployeeService	5163	EmployeeDb	/api/v1/employees/*
DepartmentService	5104	DepartmentDb	/api/v1/departments/*
SalaryService	5118	SalaryDb	/api/v1/salaries/*, /salary-components/*, /bonuses/*, /overtime/*, /payroll/*, /payslips/*
LeaveService	62417	LeaveDb	/api/v1/leaves/*
AttendanceService	64365	AttendanceDb	/api/v1/attendance/*
SupportService	5133	SupportDb	/api/v1/support-tickets/*

Ports are current development values from the project brief. S2 lists different internal ports (5001-5007); the brief values are used here. Ports are configuration items and may change per environment.

8.4 System Architecture Diagram

Figure 8.1 shows the architecture required by this SRS: frontend, API Gateway, seven services with their configured ports, Entity Framework Core and one SQL Server Express database per service. Only technologies from the project brief are shown.

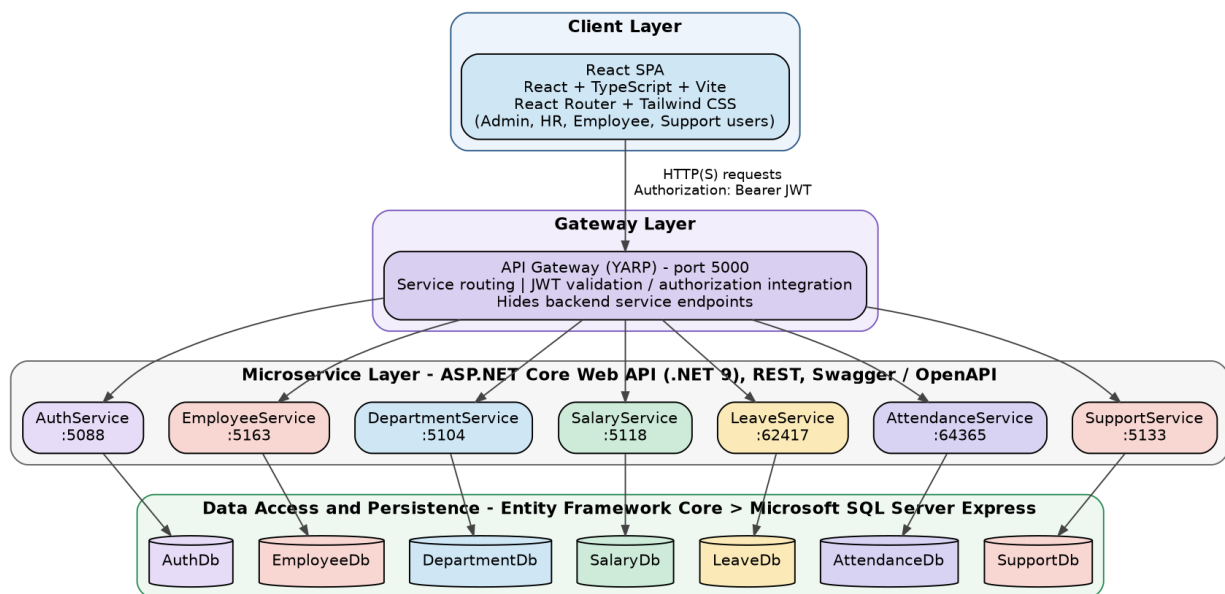


Figure 8.1: EMS system architecture (Frontend > API Gateway > Services > EF Core > SQL Server)

Internal service layering

Each service follows the layered structure below where applicable (Figure 8.2).

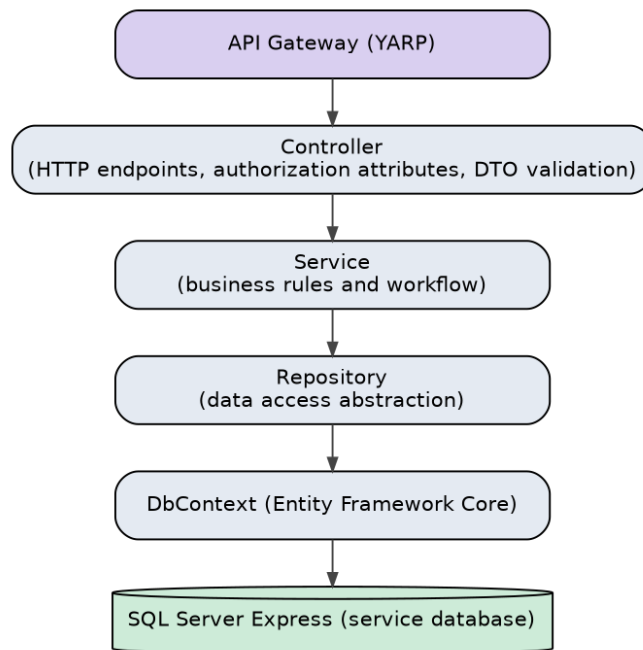


Figure 8.2: Layered structure inside each backend service

Supplied High Level Design

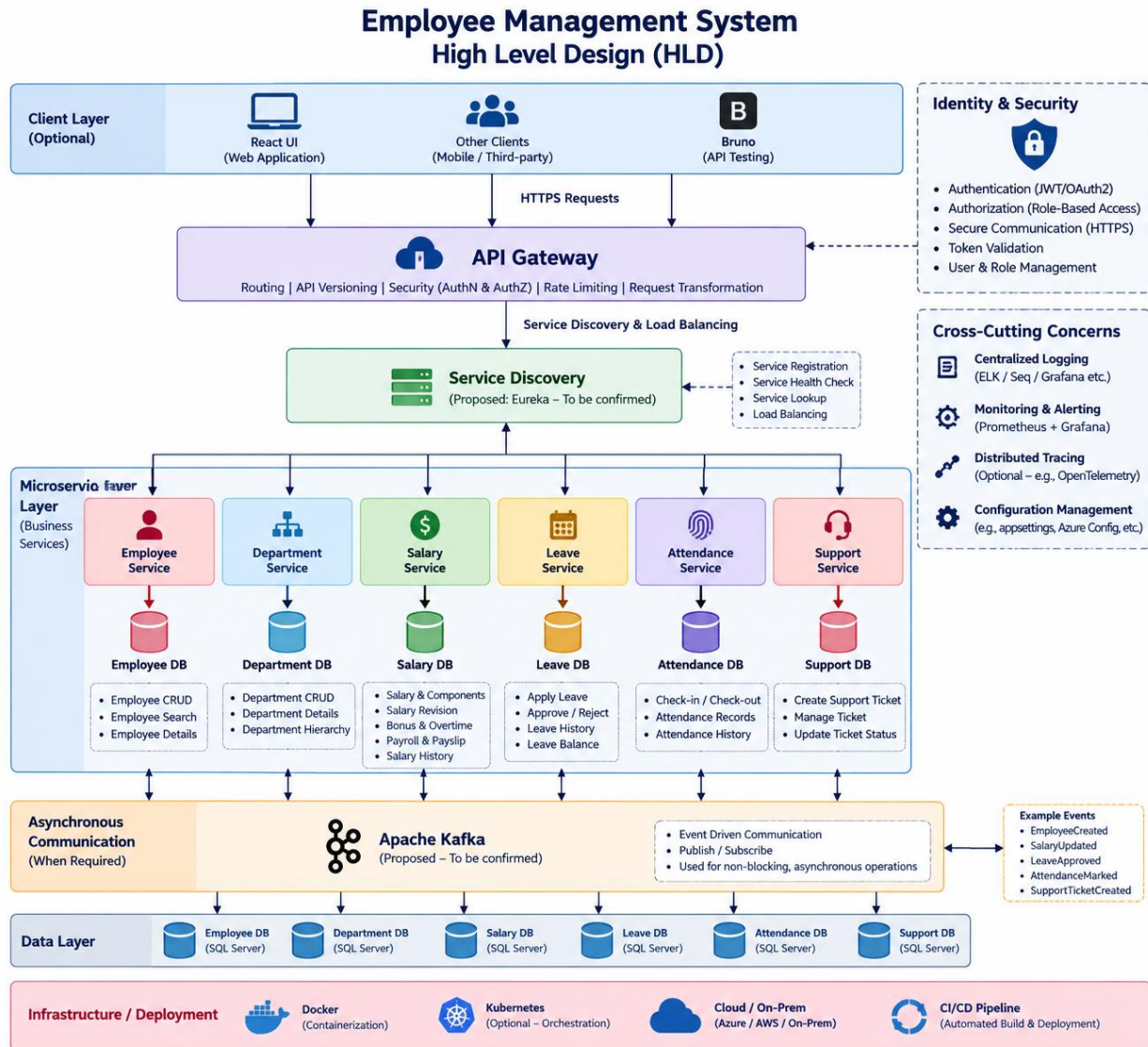


Figure 8.3: High Level Design supplied with the project (S3)

Interpretation note: the supplied HLD (Figure 8.3) shows a Service Discovery layer (Eureka), Apache Kafka, Docker/Kubernetes, CI/CD and monitoring tools. The diagram itself marks Eureka and Kafka as "Proposed - To be confirmed". These are not requirements of this SRS (see Section 21). The HLD also shows API versioning, rate limiting and request transformation at the gateway; these are not in the project brief and are treated as To be confirmed. The HLD lists the services with their own databases, which matches this SRS.

9. Technology Stack

Layer	Technology	Purpose
Frontend	React, TypeScript, Vite, React Router, Tailwind CSS	SPA user interface, routing and styling
Backend	C#, ASP.NET Core Web API, .NET 9, REST APIs	Microservice APIs
Data access	Entity Framework Core	ORM, migrations, parameterised queries
Gateway	YARP (Yet Another Reverse Proxy)	Central routing and access point
Authentication	JWT, role-based authorization, BCrypt	Identity and access control
Database	Microsoft SQL Server Express	Relational persistence (one database per service)
API documentation	Swagger / OpenAPI	Interactive API documentation

Layer	Technology	Purpose
API testing	Bruno (shown in HLD as API testing client)	Manual API testing. Use: To be confirmed.

S2 mentions PostgreSQL as an alternative database. The project brief specifies SQL Server Express, which is used in this SRS.

10. Database Requirements

10.1 Database Architecture

EMS uses the database-per-service pattern: seven databases on Microsoft SQL Server Express, each accessed only by its own service through Entity Framework Core.

Database	Owning service	Main data
AuthDb	AuthService	User accounts, hashed passwords, roles
EmployeeDb	EmployeeService	Employee records
DepartmentDb	DepartmentService	Departments
LeaveDb	LeaveService	Leave requests and decisions
AttendanceDb	AttendanceService	Attendance records
SalaryDb	SalaryService	Salary, components, revisions, bonuses, overtime, payroll, payslips
SupportDb	SupportService	Support tickets

10.2 Reasons for Separate Databases

- Loose coupling: a service can change its schema without affecting other services.
- Ownership: each service is the only writer of its data, which supports clear responsibilities.
- Independent deployment and scaling of services and their data.
- Fault isolation: a problem in one database does not directly affect another domain.
- Security: sensitive data such as credentials (AuthDb) and salary (SalaryDb) is stored separately and access can be restricted per service.

10.3 Consequences and Requirements

- There are no database-level foreign keys between databases. References such as `Leaves.EmployeeId` or `Employees.DepartmentId` are logical references validated by the application. How validation is done (inter-service call or trusted identifiers from the token): To be confirmed.
- Operations spanning services (for example, blocking department deletion when employees exist) require service-to-service or gateway-level coordination: To be confirmed.
- Each service must maintain its own EF Core migrations.
- Primary keys, unique constraints and required-field constraints must be enforced in the database as well as in application validation.
- Connection strings must be held in secure configuration (Section 13).

11. Database Design

Entities and fields below are taken from the existing SRS v2.0 (S2). No other fields are added. Data types are not specified in S2 except where noted; types are To be confirmed. Identifiers are described in S2 as UUID. All tables also carry CreatedAt and UpdatedAt as listed in S2.

11.1 Entities

AuthDb - Users

Field	Key / Constraint	Notes
UserId	PK	UUID
Username	Unique	
Email	Unique	
PasswordHash	Required	BCrypt hash
Role	Required	Admin, HR, Employee or Support
CreatedAt, UpdatedAt	-	

EmployeeDb - Employees

Field	Key / Constraint	Notes
EmployeeId	PK	UUID
UserId	FK (logical) to Users	Links to AuthDb identity
FirstName, LastName	Required	
Email	Unique	
Phone	-	
DateOfJoining	-	Not in the future (BR-EMP-002)
DepartmentId	FK (logical) to Departments	
Role	-	
EmploymentStatus	-	Allowed values: To be confirmed
CreatedAt, UpdatedAt	-	

DepartmentDb - Departments

Field	Key / Constraint	Notes
DepartmentId	PK	UUID
DepartmentName	Unique	
Description	-	
CreatedAt, UpdatedAt	-	

LeaveDb - Leaves

Field	Key / Constraint	Notes
LeaveId	PK	UUID
EmployeeId	FK (logical) to Employees	
LeaveType	Required	Allowed values: To be confirmed
StartDate, EndDate	Required	StartDate <= EndDate
Reason	-	
Status	Required	Pending, Approved, Rejected

Field	Key / Constraint	Notes
ApprovedBy, ApprovedAt	Nullable	Reviewer and time
AppliedAt	-	
CreatedAt, UpdatedAt	-	

AttendanceDb - Attendances

Field	Key / Constraint	Notes
AttendanceId	PK	UUID
EmployeeId	FK (logical) to Employees	
AttendanceDate	Required	Date; one check-in per employee per date
CheckInTime	Required	
CheckOutTime	Nullable	
Status	Required	CheckedIn, CheckedOut
CreatedAt, UpdatedAt	-	

SupportDb - SupportTickets

Field	Key / Constraint	Notes
TicketId	PK	UUID
EmployeeId	FK (logical) to Employees	Ticket creator
Title, Description	Required	
Priority	Required	Low, Medium, High
Status	Required	Open, Assigned, InProgress, Resolved, Closed
AssignedTo	Nullable	UUID of assignee
CreatedAt, UpdatedAt	-	
ResolvedAt	Nullable	

SalaryDb - EmployeeSalaries

Field	Key / Constraint	Notes
EmployeeSalaryId	PK	UUID
EmployeeId	Unique	One salary record per employee
BasicSalary	Required	Not negative
EffectiveFrom	Required	
CreatedAt, UpdatedAt	-	

SalaryDb - SalaryComponents

Field	Key / Constraint	Notes
SalaryComponentId	PK	
EmployeeId	FK	Employee salary owner
ComponentName, Amount	Required	
ComponentType	Required	Earning or Deduction
CreatedAt, UpdatedAt	-	

SalaryDb - SalaryRevisions

Field	Key / Constraint	Notes
SalaryRevisionId	PK	
EmployeeId	FK	
PreviousSalary, RevisedSalary	Required	
RevisionDate, Reason	-	
CreatedAt, UpdatedAt	-	

SalaryDb - Bonuses

Field	Key / Constraint	Notes
BonusId	PK	
EmployeeId	FK	
BonusType, Amount, BonusDate, Reason	-	Amount not negative
CreatedAt, UpdatedAt	-	

SalaryDb - Overtimes

Field	Key / Constraint	Notes
OvertimeId	PK	
EmployeeId	FK	
OvertimeDate, Hours, Rate, Amount	Required	Amount = Hours x Rate
Status	Required	Pending, Approved, Rejected
ApprovedBy, ApprovedAt	Nullable	
CreatedAt, UpdatedAt	-	

SalaryDb - Payrolls

Field	Key / Constraint	Notes
PayrollId	PK	
PayrollMonth, PayrollYear	Required	Integers
GeneratedAt	-	
Status	-	Allowed values: To be confirmed
CreatedAt, UpdatedAt	-	

SalaryDb - PayrollItems

Field	Key / Constraint	Notes
PayrollItemId	PK	
PayrollId	FK to Payrolls	
EmployeeId	FK	
BasicSalary, TotalComponents, TotalBonus, OvertimeAmount, TotalDeductions, NetSalary	Required	Monetary values
CreatedAt, UpdatedAt	-	

SalaryDb - Payslips

Field	Key / Constraint	Notes
PayslipId	PK	
PayrollId	FK to Payrolls	
EmployeeId	FK	
PayslipNumber	Unique	
PayslipDate	-	
BasicSalary, TotalComponents, TotalBonus, OvertimeAmount, TotalDeductions, NetSalary	Required	Copied from the payroll item at generation
CreatedAt, UpdatedAt	-	

11.2 Relationships

Relationship	Cardinality	Type
Users - Employees	1 : 1	Logical (cross-database)
Departments - Employees	1 : N	Logical (cross-database)
Employees - Leaves	1 : N	Logical (cross-database)
Employees - Attendances	1 : N	Logical (cross-database)
Employees - SupportTickets	1 : N	Logical (cross-database)
Employees - EmployeeSalaries	1 : 1	Logical (cross-database)
EmployeeSalaries - SalaryComponents / SalaryRevisions / Bonuses / Overtimes	1 : N	Within SalaryDb by EmployeeId (S2 ERD; exact FK target To be confirmed)
Payrolls - PayrollItems	1 : N	Foreign key within SalaryDb
Payrolls - Payslips	1 : N	Foreign key within SalaryDb

11.3 ER Diagram

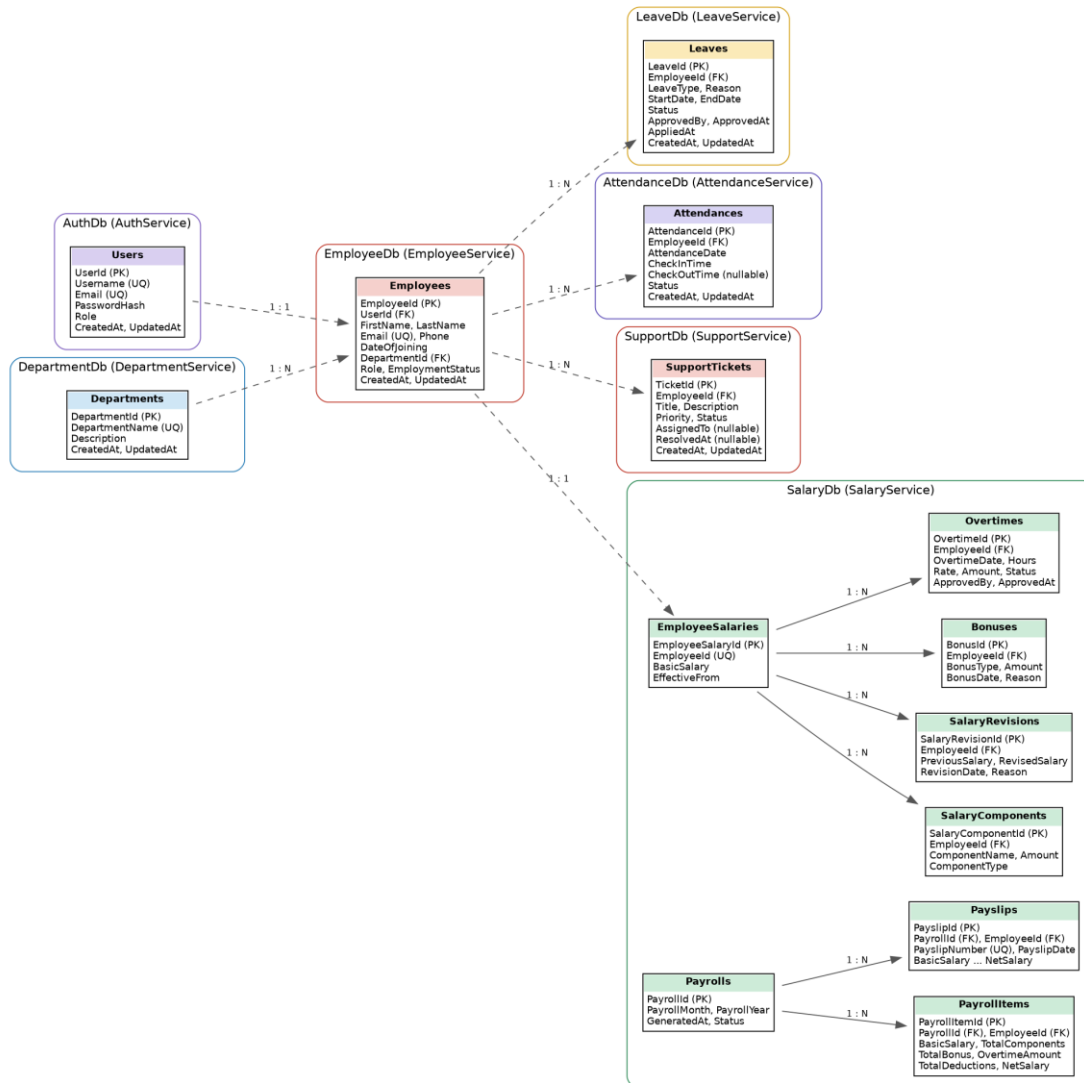


Figure 11.1: ER diagram (regenerated from S2 entities; dashed lines are logical cross-database references)

11.4 Primary Keys, Foreign Keys and Constraints

- Primary keys: one UUID primary key per table as listed in 11.1.
- Foreign keys: real database foreign keys only within SalaryDb (PayrollItems and Payslips to Payrolls; salary child tables by EmployeeId, To be confirmed). All other references are logical.
- Unique constraints: Users.Username, Users.Email, Employees.Email, Departments.DepartmentName, EmployeeSalaries.EmployeeId, Payslips.PayslipNumber. Unique attendance per employee and date (Attendances EmployeeId + AttendanceDate) is required by BR-ATT-001; the composite index: To be confirmed.
- Value constraints: salary and bonus amounts not negative; leave StartDate <= EndDate; checkout after check-in; enumerated status/priority/type values as listed.
- Indexes on frequently queried identifiers (EmployeeId, DepartmentId, Status): To be confirmed.

11.5 Normalization

Each table describes one entity, with atomic fields and no repeating groups (1NF). Non-key attributes depend on the whole key (2NF, single-column keys) and not on other non-key attributes (3NF). Cross-service duplication is limited to identifiers (UserId, EmployeeId, DepartmentId). Payslips and PayrollItems repeat monetary totals

intentionally: they are immutable snapshots of a finalised payroll and must not change when salary data later changes. This is a deliberate exception to strict normalization.

12. API Requirements

12.1 General

- All requests from the frontend go through the API Gateway (http://localhost:5000 in development). Route prefix /api/v1 follows S2 and is To be confirmed against the implementation.
- Payloads use JSON. Protected requests carry Authorization: Bearer <JWT>.
- Each service publishes Swagger / OpenAPI documentation.
- S2 also requires an X-Correlation-ID header for tracing; its use is To be confirmed (Section 18).
- In tables below, "Authenticated" means any valid token; "Owner" means the record belongs to the caller. Endpoints with route "To be confirmed" are required by this SRS but not listed in S2.
- Error bodies follow Section 17. Only the main error codes are listed per endpoint.

12.2 Authentication APIs (AuthService)

Method	Endpoint	Description	Authorization	Success Response	Error Response
POST	/api/v1/auth/register	Public registration. Role is always Employee.	Anonymous	201 Created	400, 409
POST	/api/v1/auth/login	Authenticate and return JWT.	Anonymous	200 OK (token)	400, 401
POST	To be confirmed	Admin creates/manages Admin, HR or other privileged accounts.	Admin	201 Created / 200 OK	400, 401, 403, 409
-	None (client-side)	Logout: frontend clears token. Server logout: To be confirmed.	Authenticated	Not applicable	Not applicable

12.3 Employee APIs (EmployeeService)

Method	Endpoint	Description	Authorization	Success Response	Error Response
GET	/api/v1/employees	List employees.	Admin, HR (Support read: TBC)	200 OK	401, 403
POST	/api/v1/employees	Create employee.	Admin, HR	201 Created	400, 401, 403, 409
GET	/api/v1/employees/{id}	Get employee by id.	Admin, HR	200 OK	401, 403, 404
PUT	/api/v1/employees/{id}	Update employee.	Admin, HR	200 OK / 204	400, 401, 403, 404, 409
DELETE	/api/v1/employees/{id}	Delete employee.	Admin (HR: TBC)	204 No Content	401, 403, 404
GET	/api/v1/employees/search	Search by term (name, role, email).	Admin, HR	200 OK	401, 403
GET	/api/v1/employees/me	Get own profile.	Authenticated	200 OK	401, 404

12.4 Department APIs (DepartmentService)

Method	Endpoint	Description	Authorization	Success Response	Error Response
GET	/api/v1/departments	List departments.	Authenticated	200 OK	401
POST	/api/v1/departments	Create department.	Admin, HR	201 Created	400, 401, 403, 409
GET	/api/v1/departments/{id}	Get department by id.	Authenticated	200 OK	401, 404
PUT	/api/v1/departments/{id}	Update department.	Admin, HR	200 OK / 204	400, 401, 403, 404, 409
DELETE	/api/v1/departments/{id}	Delete department (refused if employees assigned).	Admin, HR	204 No Content	401, 403, 404, 409 (TBC)
GET	To be confirmed	List employees of a department.	Admin, HR	200 OK	401, 403, 404

12.5 Leave APIs (LeaveService)

Method	Endpoint	Description	Authorization	Success Response	Error Response
POST	/api/v1/leaves	Apply for leave (status Pending).	Authenticated	201 Created	400, 401, 409
GET	/api/v1/leaves/{id}	Get leave details.	Owner, HR, Admin	200 OK	401, 403, 404
GET	/api/v1/leaves/employee/{empId}	List leaves of an employee.	Owner, HR, Admin	200 OK	401, 403
GET	/api/v1/leaves/{empId}/history	Leave history of an employee.	Owner, HR, Admin	200 OK	401, 403
PUT	/api/v1/leaves/{id}/approve	Approve leave (not own).	HR, Admin	200 OK	400, 401, 403, 404, 409
PUT	/api/v1/leaves/{id}/reject	Reject leave (not own).	HR, Admin	200 OK	400, 401, 403, 404, 409

12.6 Attendance APIs (AttendanceService)

Method	Endpoint	Description	Authorization	Success Response	Error Response
POST	/api/v1/attendance/check-in	Record check-in.	Authenticated	201 Created	401, 409
PUT	/api/v1/attendance/{id}/check-out	Record check-out.	Owner	200 OK	400, 401, 403, 404, 409
GET	/api/v1/attendance/{id}	Get attendance record.	Owner, HR, Admin	200 OK	401, 403, 404
GET	/api/v1/attendance/employee/{empId}	Attendance history of an employee.	Owner, HR, Admin	200 OK	401, 403
GET	/api/v1/attendance	All attendance records.	Admin, HR	200 OK	401, 403
GET	To be confirmed	Weekly / monthly summary (or computed in frontend).	Owner; Admin, HR	200 OK	401, 403

12.7 Salary APIs (SalaryService)

Method	Endpoint	Description	Authorization	Success Response	Error Response
POST	/api/v1/salaries	Create base salary.	Admin, HR	201 Created	400, 401, 403, 409
GET	/api/v1/salaries/{empId}	Get salary.	Owner, Admin, HR	200 OK	401, 403, 404
PUT	/api/v1/salaries/{empId}	Update salary.	Admin, HR	200 OK / 204	400, 401, 403, 404
POST	/api/v1/salaries/{empId}/components	Add component.	Admin, HR	201 Created	400, 401, 403, 404
GET	/api/v1/salaries/{empId}/components	List components.	Owner, Admin, HR	200 OK	401, 403, 404
PUT	/api/v1/salary-components/{id}	Update component.	Admin, HR	200 OK	400, 401, 403, 404
POST	/api/v1/salaries/{empId}/revisions	Record revision.	Admin, HR	201 Created	400, 401, 403, 404
GET	/api/v1/salaries/{empId}/revisions	Revision history.	Admin, HR (Owner: TBC)	200 OK	401, 403, 404
POST	/api/v1/salaries/{empId}/bonuses	Grant bonus.	Admin, HR	201 Created	400, 401, 403, 404
GET	/api/v1/salaries/{empId}/bonuses	List bonuses.	Owner, Admin, HR	200 OK	401, 403, 404
PUT	/api/v1/bonuses/{id}	Update bonus.	Admin, HR	200 OK	400, 401, 403, 404
POST	/api/v1/overtime	Submit overtime (Pending).	Authenticated	201 Created	400, 401
GET	/api/v1/overtime/{empId}	Overtime records of employee.	Owner, Admin, HR	200 OK	401, 403, 404
PUT	/api/v1/overtime/{id}/approve	Approve overtime.	Admin, HR	200 OK	401, 403, 404, 409
PUT	/api/v1/overtime/{id}/reject	Reject overtime.	Admin, HR	200 OK	401, 403, 404, 409
POST	/api/v1/payroll	Generate monthly payroll.	Admin, HR	201 Created	400, 401, 403, 409

Method	Endpoint	Description	Authorization	Success Response	Error Response
GET	/api/v1/payroll	List payrolls.	Admin, HR	200 OK	401, 403
GET	/api/v1/payroll/{id}	Get payroll.	Admin, HR	200 OK	401, 403, 404
GET	/api/v1/payroll/{id}/items	Get payroll items.	Admin, HR	200 OK	401, 403, 404
POST	/api/v1/payslips/{payId}/{emplId}	Generate payslip.	Admin, HR	201 Created	400, 401, 403, 404, 409
GET	/api/v1/payslips	List payslips (own for Employee).	Owner, Admin, HR	200 OK	401, 403
GET	/api/v1/payslips/{id}	Get payslip.	Owner, Admin, HR	200 OK	401, 403, 404

12.8 Support APIs (SupportService)

Method	Endpoint	Description	Authorization	Success Response	Error Response
POST	/api/v1/support-tickets	Create ticket (status Open).	Authenticated	201 Created	400, 401
GET	/api/v1/support-tickets	List all tickets.	HR, Support, Admin	200 OK	401, 403
GET	/api/v1/support-tickets/{id}	Get ticket.	Owner, HR, Support, Admin	200 OK	401, 403, 404
PUT	/api/v1/support-tickets/{id}	Assign ticket / update status.	Support, Admin (HR: TBC)	200 OK	400, 401, 403, 404
GET	/api/v1/support-tickets/employee/{id}	Tickets of an employee.	Owner, HR, Support, Admin	200 OK	401, 403

S2 writes the last route as `/api/v1/supporttickets/employee/{id}`; the hyphenated form is used here for consistency and must be confirmed. Ticket history has no dedicated endpoint (FR-SUP-008).

12.9 Request and Response Content

Request and response field lists are derived from the entity fields in Section 11. Exact DTO field names are To be confirmed against the Swagger definitions of each service. Examples of the shape of an error body are in Section 17.

13. Security Requirements

ID	Area	Requirement
SR-001	JWT authentication	Protected APIs must accept only valid, signed, unexpired JWTs sent as Authorization: Bearer. Claims: user id, role, email, employeeId (S2).
SR-002	JWT expiration	Tokens must have a limited lifetime and expired tokens are rejected with 401. Lifetime value and any refresh mechanism: To be confirmed.
SR-003	Role-based authorization	Each endpoint must check the role claim against permitted roles (Section 4.1). Checks at both gateway and service are recommended; gateway role enforcement: To be confirmed.
SR-004	Password hashing	Passwords must be hashed with BCrypt; plaintext passwords must never be stored or logged.
SR-005	Password security	Passwords must satisfy complexity rules (Section 16). Login errors must not reveal whether username or password was wrong.
SR-006	Protected endpoints	All endpoints except register and login require authentication.
SR-007	Registration security	Public registration always creates Employee accounts. Privileged accounts are created only by Admin.
SR-008	Input validation	All inputs must be validated on the server; client validation is only a convenience.
SR-009	Unauthorized access	Missing/invalid token returns 401.
SR-010	Forbidden access	Valid token with insufficient role or ownership returns 403.
SR-011	SQL injection prevention	Data access must use EF Core LINQ or parameterised queries; no string-concatenated SQL.

ID	Area	Requirement
SR-012	Secure secrets	JWT signing key and database connection strings must be stored outside source code (e.g. environment variables, user secrets or a secrets manager; choice To be confirmed).
SR-013	HTTPS	Production deployments must use HTTPS. Development uses HTTP on localhost. Certificate and TLS version: To be confirmed.
SR-014	CORS	The gateway/services must allow only the frontend origin (S2 mentions CORS policy at gateway). Allowed origins: To be confirmed.
SR-015	Data exposure	Responses must not contain password hashes; salary and payslip data must be limited to owner, Admin and HR.

14. Business Rules

ID	Rule	Source
BR-AUTH-001	Public registration creates an Employee account by default; the caller cannot choose Admin or another privileged role.	S1
BR-AUTH-002	Admin/HR accounts are created or managed only through authorized administrative functionality.	S1
BR-GEN-001	Users may access only functionality permitted for their role.	S1
BR-GEN-002	Required fields must be validated on every create/update operation.	S1
BR-EMP-001	Each employee must have a unique email address.	S2
BR-EMP-002	Date of joining cannot be in the future.	S2
BR-EMP-003	Only authorized roles (Admin, HR) can manage employees.	S1, S2
BR-EMP-004	Deleting an employee: handling of related history (archived per S2) is To be confirmed.	S2
BR-DEP-001	Department names must be unique.	S2
BR-DEP-002	A department cannot be deleted while employees are assigned to it.	S1, S2
BR-SAL-001	Salary cannot be negative. S2 requires base salary to be positive and non-zero; the stricter rule applies to base salary unless the team confirms otherwise.	S1, S2
BR-SAL-002	Only authorized roles (Admin, HR) can create or modify salary data.	S1
BR-SAL-003	Salary revisions are append-only and record previous salary, revised salary, date and reason.	S2
BR-SAL-004	Overtime amount = Hours x Rate; overtime must be Approved before it is included in payroll.	S2
BR-SAL-005	Net salary = Basic + Earnings + Bonuses + Overtime - Deductions.	S2
BR-SAL-006	Finalised payroll items and payslips are immutable. Unlocking for recalculation requires Admin action (mechanism To be confirmed).	S2
BR-LEAVE-001	Leave StartDate must be on or before EndDate.	S2
BR-LEAVE-002	An employee cannot have overlapping leave periods.	S2
BR-LEAVE-003	A user cannot approve or reject their own leave.	S1, S2
BR-LEAVE-004	Only HR and Admin can approve or reject leave. Only Pending leave can be decided.	S1
BR-ATT-001	Only one check-in per employee per calendar date.	S2
BR-ATT-002	Check-out time must be after check-in time.	S2
BR-SUP-001	Ticket status flow: Open, Assigned, InProgress, Resolved, Closed. Whether Open may move directly to InProgress: To be confirmed.	S2
BR-SUP-002	Only Support or Admin users can take ownership of assigned tickets.	S2

15. UI Requirements

The frontend is a React + TypeScript single-page application built with Vite, React Router and Tailwind CSS. UI elements below are Required unless marked. Screens named in S2/S4 but outside S1 are excluded (Section 3.2).

ID	Screen / Requirement	Content
UI-001	Login and Registration	Login form and public registration form. Registration does not show a role selector. Error messages for invalid credentials.
UI-002	Role-based dashboard	After login, redirect to a dashboard by role (Admin, HR, Employee, Support) with navigation limited to permitted modules (S4).
UI-003	Employee module	Searchable employee list, add/edit forms, details view, delete confirmation. Visible to Admin, HR.
UI-004	Department module	Department list with employee counts, create/edit forms, delete with restriction message.
UI-005	Salary and payroll (Admin/HR)	Salary assignment, component management, revision recording and history, bonus, overtime approval queue, payroll generation, payslip generation.
UI-006	Salary (Employee)	Own salary, components, overtime submission, revision history, payslips.
UI-007	Leave module	Leave application form, own leave history with status, approval queue with approve/reject for HR/Admin.
UI-008	Attendance module	Check-in / check-out buttons, current status, daily hours, history, weekly/monthly view; team view for Admin/HR.
UI-009	Support module	Ticket form with priority; own ticket list; ticket queue with assignment and status change for Support/Admin; ticket view for HR.
UI-010	Profile	Own profile view with limited self-service editing (editable fields To be confirmed).
UI-011	Route protection	Protected routes redirect unauthenticated users to login; role-restricted routes show an access-denied message.
UI-012	Feedback	Inline validation errors, loading and empty states, success and error notifications.
UI-013	Logout	Clears token and returns to login page.
UI-014	Not required	Forgot password, notification centre, Manager dashboard and project allocation screens shown in S2/S4 are Future Enhancements.

16. Validation Requirements

Area	Validation
Registration	Username, email, password required. Email format valid. Username and email unique. Password: minimum 8 characters with uppercase, lowercase, digit and special character (S2). Role must not be accepted from the caller.
Login	Identifier and password required.
Employee	First name, last name, email required; email unique and valid; date of joining not in the future; department exists; phone format To be confirmed.
Department	Name required and unique; description optional.
Salary	Employee exists; basic salary not negative (positive per S2); effective date required; component type Earning or Deduction; amounts not negative; bonus amount not negative.
Overtime	Date required; hours greater than zero; rate not negative; only Pending records can be approved or rejected.
Payroll	Month 1-12, valid year; payroll for the same period is not created twice (To be confirmed).
Leave	Type, start date, end date required; start <= end; no overlap; only Pending leave can be decided; reviewer not the applicant.
Attendance	One check-in per date; check-out only after an open check-in; check-out time after check-in time.
Support	Title and description required; priority in Low, Medium, High; status transitions per BR-SUP-001; assignee has Support or Admin role.
General	Identifiers must be valid UUIDs. Validation is performed on the server (data annotations or equivalent; library To be confirmed) and mirrored in the UI.

17. Error Handling Requirements

Status	Situation	Required behaviour
400 Bad Request	Invalid input or business-rule violation	Descriptive message; UI highlights the field.
401 Unauthorized	Missing, invalid or expired token; invalid login	Frontend clears session and redirects to login.
403 Forbidden	Role or ownership not permitted	Access-denied message; session kept.
404 Not Found	Resource does not exist	Friendly not-found state.
409 Conflict	Duplicate value or state conflict (duplicate email, second check-in, overlapping leave, decided request)	Conflict message shown to the user.
500 Internal Server Error	Unhandled exception	Generic message without stack trace or internal detail; error logged with a trace identifier.

- Errors should use one consistent JSON structure. S2 specifies RFC 7807 ProblemDetails (type, title, status, detail, instance, traceId); adoption in all services: To be confirmed.
- Error messages must not reveal passwords, tokens, connection strings or stack traces.
- The frontend must handle network errors and gateway/service unavailability with a clear message.

18. Logging Requirements

ID	Requirement
LOG-001	Each service and the gateway must log requests, unhandled exceptions and authentication failures.
LOG-002	Passwords, tokens, password hashes and signing keys must never be written to logs.
LOG-003	Important business events should be logged with user id and time: salary create/update/revision, payroll generation, leave approval/rejection, employee deletion, ticket assignment and status change, privileged account creation.
LOG-004	S2 requires an X-Correlation-ID header so a request can be traced across gateway and services. Implementation: To be confirmed.
LOG-005	Log levels (information, warning, error) must be configurable per environment.
LOG-006	Log destination and tooling: To be confirmed. S3 lists ELK, Seq or Grafana only as examples of centralised logging.

19. Assumptions and Dependencies

19.1 Assumptions

- Each user has one role (Admin, HR, Employee or Support).
- Each employee has one linked user account (Users 1 : 1 Employees, per S2).
- Frontend, gateway and services run in the same environment and can reach one another.
- Server clocks are reasonably synchronised so that token expiry and timestamps are consistent.
- Salary values use a single currency (currency To be confirmed).
- The first Admin account is created by a seeding or setup step (mechanism To be confirmed).

19.2 Dependencies

- .NET 9 SDK/runtime, ASP.NET Core, Entity Framework Core, YARP.
- Microsoft SQL Server Express instance with the seven databases.
- Node.js and Vite tooling for the frontend build.
- A BCrypt library and a JWT library for .NET (package names To be confirmed).
- Swagger / OpenAPI tooling for API documentation.
- A modern web browser.

20. System Constraints

- Technology stack is fixed to the technologies listed in Section 9.
- The frontend must access the backend only through the API Gateway.
- Each service must use its own database; direct cross-database queries are not allowed.
- Service ports are the configured development values (Section 8.3).
- SQL Server Express limits (database size, memory) apply to all seven databases.
- All business APIs other than register and login require JWT authentication.
- Financial records that are finalised (payroll items, payslips) must not be altered.
- The project is a college/training project; production deployment, hosting and licensing are To be confirmed.

21. Future Enhancements

The following items are not required for the current scope. Items marked (S2), (S3) or (S4) appear in the supporting artifacts.

ID	Enhancement	Note
FE-01	Forgot / reset password flow (S4)	Requires a secure reset mechanism and possibly e-mail delivery.
FE-02	Notification module including e-mail notifications for leave, salary and ticket events (S4)	Would need an event/notification design.
FE-03	Manager role and Manager dashboard for team leave/attendance approval (S2, S4)	Would change role model and approval rules.
FE-04	Project and workload allocation (S2)	New module and data model.
FE-05	Asynchronous event messaging such as Apache Kafka (S3, proposed)	Example events shown in S3: EmployeeCreated, SalaryUpdated, LeaveApproved, AttendanceMarked, SupportTicketCreated.
FE-06	Service discovery such as Eureka (S3, proposed)	Not required while YARP static routes are used.
FE-07	Containerisation (Docker), orchestration (Kubernetes) and CI/CD (S3)	Deployment approach to be decided.
FE-08	Centralised logging, monitoring and distributed tracing (S3)	Tools to be decided.
FE-09	Gateway rate limiting, API versioning and request transformation (S3)	Not in project brief.
FE-10	Leave balance tracking and leave types configuration	Referenced in S2/S3 UI but not in S1.
FE-11	Refresh tokens and server-side token revocation	Improves logout and session security.
FE-12	Reports and exports (payroll, attendance, leave)	Not mentioned in sources; suggested enhancement.

22. Acceptance Criteria

The system is accepted when the criteria below are met and demonstrated by tests. Items marked To be confirmed in this SRS must first be decided.

ID	Criteria	Related requirements
AC-001	A public user can register and the account has the Employee role. Registration with an Admin/HR role is not possible.	FR-AUTH-001, 002; BR-AUTH-001
AC-002	A registered user can log in and receives a JWT; wrong credentials return 401; protected APIs return 401 without a token and 403 for wrong roles.	FR-AUTH-004 to 009
AC-003	Passwords in AuthDb are BCrypt hashes.	FR-AUTH-010

ID	Criteria	Related requirements
AC-004	Admin/HR can add, update, search and view employees; Employee can view only own profile; delete works for the authorized role.	FR-EMP-001 to 009
AC-005	Departments can be created, viewed, updated and deleted; duplicate names are rejected; a department with employees cannot be deleted.	FR-DEP-001 to 007
AC-006	Admin/HR can create and revise salary, add components and bonuses, and view revision history; negative values are rejected; Employee cannot modify salary and sees only own data.	FR-SAL-001 to 007, 011
AC-007	Overtime follows Pending to Approved/Rejected; only approved overtime enters payroll; payroll and payslips can be generated and viewed by authorized users.	FR-SAL-008 to 010
AC-008	Employee can apply for leave and view history; invalid dates and overlaps are rejected; HR/Admin can approve or reject; nobody can decide their own leave.	FR-LEAVE-001 to 007
AC-009	Employee can check in once per day and check out; check-out before check-in is rejected; working hours are shown; Employees see only own attendance and Admin/HR see all.	FR-ATT-001 to 006
AC-010	Employee can create tickets; HR/Support can view tickets; Support/Admin can assign and move tickets through InProgress, Resolved and Closed.	FR-SUP-001 to 008
AC-011	The frontend reaches all services only through the gateway on port 5000; service ports are not used by the frontend.	FR-GW-001 to 003
AC-012	Each service has Swagger documentation and its own database.	NFR-MAIN-003, 004; Section 10
AC-013	Validation, 400/401/403/404/409 handling and UI error messages behave as in Sections 16 and 17.	Sections 16, 17
AC-014	Performance targets in Section 7 are tested; results are recorded (pass/fail against S2 targets).	NFR-PERF-001, 002

Appendix A. Diagram Specifications

For each diagram the table states what it must show. Diagrams supplied with the project are inserted where they exist; others are placeholders to be drawn by the team.

No.	Diagram	What it must show	Availability
A1	System Architecture Diagram	Frontend, API Gateway (5000), seven services with ports, seven databases, and request flow Frontend > Gateway > Service > EF Core > SQL Server.	Provided as Figure 8.1 (simplified, brief-only) and Figure 8.2 (service layering). The supplied HLD (Figure 8.3) also includes proposed components.
A2	Use Case Diagram	Actors: Public User, Admin, HR, Employee, Support. Use cases per module (register, login, manage employees, manage departments, manage salary, apply/approve leave, check-in/out, create/assign tickets). Include relationship showing approval excluding own leave.	Placeholder
A3	ER Diagram	Entities, keys and relationships of the seven databases.	Regenerated (Figure 11.1). The supplied ER_diagram.png was blank.
A4	Class Diagram	Per service: controller, service interface/class, repository, entity, DTO and DbContext classes and their dependencies. Class names To be confirmed from the code.	Placeholder
A5	Sequence Diagram	At minimum: login (Frontend, Gateway, AuthService, AuthDb, JWT returned) and leave approval (Frontend, Gateway, LeaveService, LeaveDb, self-approval check). Optionally check-in and payroll generation.	Placeholder
A6	Activity Diagram	Flows for authentication, leave, attendance, support ticket and salary/payroll with decisions and error paths.	Supplied User Flow diagram (Figure A.1) covers this at overview level.
A7	Data Flow Diagram	Level 0: external entities (users by role), the EMS process and data stores. Level 1: processes per module and the seven data stores, showing which data flows through the gateway.	Placeholder
A8	API Gateway Flow Diagram	Request path: client, gateway, JWT validation, route matching, service, response; branches for 401, 403 and unavailable service.	Placeholder

A.1 Placeholders

- Figure A.2 - Use Case Diagram (to be inserted)
- Figure A.3 - Class Diagram (to be inserted)
- Figure A.4 - Sequence Diagrams: login and leave approval (to be inserted)
- Figure A.5 - Data Flow Diagram, Level 0 and Level 1 (to be inserted)
- Figure A.6 - API Gateway Flow Diagram (to be inserted)

A.2 User Flow Diagram (supplied)

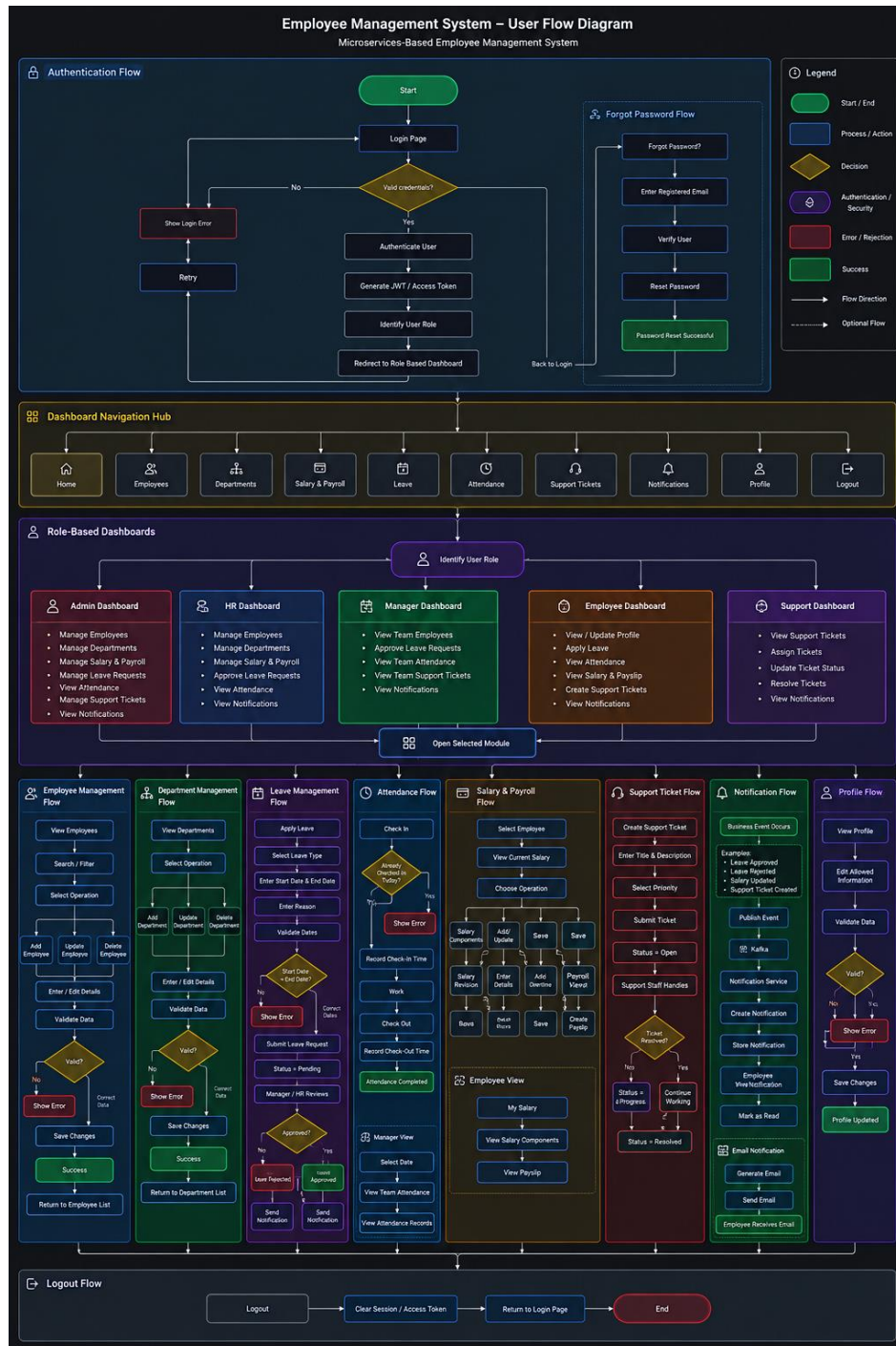


Figure A.1: User Flow Diagram supplied with the project (S4)

Interpretation note: the diagram includes a Forgot Password flow, a Manager dashboard, and a Notification flow (Kafka, e-mail). These are outside the required scope (Section 3.2, Section 21). Login, role-based dashboards for Admin, HR, Employee and Support, and the employee, department, leave, attendance, salary, support and logout flows correspond to requirements in Section 6.

Appendix B. Reconciliation with Source Material

Differences between the project brief (S1), the existing SRS v2.0 (S2) and the diagrams (S3, S4). S1 was treated as authoritative.

Topic	Other source says	Treatment in this SRS
Roles	S2/S4 include a Manager role.	Not a role; leave and overtime approval assigned to HR/Admin. Future Enhancement FE-03.
Registration	S2: registration is admin-only and self-registration restricted.	S1 rule used: public registration creates Employee; privileged accounts by Admin only.
Database	S2: PostgreSQL or SQL Server 2022.	SQL Server Express.
Service ports	S2: 5001-5007.	S1 ports used (Section 8.3).
Number of services	S2: 7 services + gateway.	Consistent; Section 8.
Project management	S2: FR-PRJ, BR-PRJ, project screen.	Out of scope.
Forgot password / notifications	S4 only.	Future Enhancement.
Kafka / Eureka / Docker / Kubernetes / CI-CD	S3, some marked "Proposed - To be confirmed".	Not required; Future Enhancement.
Support assignment role	S2 matrix: Support and Admin; S1: HR/Support can view tickets.	HR view; Support/Admin assign and update; HR assignment To be confirmed.
Employee delete	S2 FR-EMP-007: Admin; permission matrix: Admin and HR.	Admin; HR To be confirmed.
Leave balance	S2/S3 mention balance indicators.	Not in S1; To be confirmed / Future Enhancement.
Base salary rule	S2: positive non-zero. S1: not negative.	Both stated; stricter rule used pending confirmation.
Logout	S4 shows clearing session/token; no endpoint in S2.	Client-side logout; server-side To be confirmed.
Support route	S2: /supporttickets/employee/{id} (inconsistent).	Hyphenated route used; to be confirmed.
ER diagram file	ER_diagram.png contains no diagram.	Diagram regenerated from S2 entities; please supply the original if it exists.
Token / password details	S2: HMAC-SHA256 or RSA; TLS 1.3; complexity rules.	Complexity rules adopted; algorithm, lifetime and TLS version To be confirmed.